

blog_004 : EnKF を軽くする ROM — POD で代表点を「データから」選ぶ

読む前に：入力から最終結果まで

目的は、第 3 回の **EnKF** で毎回必要になる予報を軽くし、少数の温度状態から全温度場・変位まで復元することです。この記事では、既存の **OpenFOAM** 温度履歴を入力にして、次の順にモデルを作ります。

手順	入力 → 出力	担当する実装
POD	各セルの温度履歴 → 各セルの平均場・空間モード	run/select_points_qdeim.py
Q-DEIM	空間モード → 5 代表セル	同上
熱 ROM の校正	5 セルの時刻歴 → 熱容量・結合・放熱係数	run/build_calibrate_rom.py
予報・同化	各員の 5 温度・発熱倍率・放熱係数 → 更新状態	dacore/rom_general.py, dacore/enkf.py
全場復元	更新後 5 温度 → モード係数 → 全セル温度	gappy-POD (§5)
変位復元	モード係数 → 変位	事前計算した FrontISTR モード応答 (§7)

5 代表点は実際のセンサ 5 個を意味しません。代表点は ROM の状態、観測点はその状態を補正するための測定位置です。同化する観測の数と位置は別に選びます。また、この ROM は POD で熱方程式を直接射影したものではなく、POD/Q-DEIM で選んだ点に集中定数モデルを校正したものです。

理論を追う順は §2 の POD → §3 の手計算 → §4 の点選択 → §5 の全場復元 → §6 の時間発展 → §7 の変位。§2-5 のクリロフ法は補足です。主筋を先に読む場合は §3 へ進み、後から戻れます。式展開は省略せず残しています。

前回までの OI (blog_002) と EnKF (blog_003) は、予報モデルを毎サイクル回すのが前提でした。ところが本物の予報モデルは **OpenFOAM CHT + FrontISTR** の連成解析。1 サイクル回すだけで分オーダーかかり、EnKF のように何十もの分身 × 何サイクルも回すと、現実的な時間で終わりません。

そこでこの回は、予報モデルだけを軽い **ROM** (縮約モデル) に置き換える話です。ポイントは 1 つ:

ROM のノード (代表点) を「勘」で置かず、POD でデータから系統的に選ぶ。

「POD でモードを出す → Q-DEIM で代表点を選ぶ → その点で ROM を組んで校正 → EnKF/OI の予報を高速化」という流れを、数式 → 小さな行列の手計算 → 実データの図の順で追います。



1. なぜ ROM が要るか — EnKF は実ソルバだと重すぎる
EnKF (blog_003) は、分身 (アンサンブル) 1 つ 1 つについて予報モデルを前進させます。ざっくり計算量 $\propto$ (分身の数) $\times$ (サイクル数) $\times$ (予報モデル 1 回のコスト) です。

- 実ソルバ (CHT + FrontISTR) は 1 サイクル (60 秒ぶん) でも分オーダー。
- これを 分身 50~200 個 $\times$ 10 サイクル回すと、EnKF 1 回で数時間~数日。

一方、この記事で作る **POD 選定 5 点 ROM** は、0 → 600 秒の前進積分が約 31 ミリ秒 (実測)。この単一 ROM の計測値は、全場復元・変位評価・図生成を含む EnKF 一式の計算時間ではありません。予報モデルを **ROM** に差し替えるだけで、同じ EnKF/OI が桁違いに軽くなる、というのが狙いです。

1-1. 大事な注意 — ROM は CFD を「本番ループ」でだけ置き換える

「ROM にしたらもう OpenFOAM は要らない?」と思いますが、そうではありません。処理は オフライン (準備) と オンライン (本番) に分かれます:

フェーズ	何をやる	OpenFOAM(CHT)	回数
オフライン (準備)	CHT でスナップショット生成 → POD → Q-DEIM → 校正	必須 (ROM の材料)	1 回だけ
オンライン (本番)	軽い ROM で EnKF/OI を回して温度・ $Q \cdot h$ を推定	使わない (ROM 31ms)	何十~何百回

- **ROM は CFD の” 蒸留物 ”**: モードも代表点も係数 C, K, h も、すべて **OpenFOAM** の結果から作られます。CFD が無ければ ROM は存在しない。ROM は 3D 連成解析の代役であって、独立した物理ソルバではありません。
- つまり「高価な **CFD** を一度だけ払い → その後は安価なリアルタイム能力を得る」という前払い構造。**OpenFOAM** のメリットが消えたのではなく、ROM という形に凝縮して前倒しただけです。
- 有効範囲に注意: ROM は訓練データが覆った範囲でしか正しくない (外挿は苦手)。形状・境界条件・加熱シナリオが大きく変わると無効になり、CFD を回して作り直し/再校正が要ります。

記号の読み替えと行列サイズ

§2~5 の u は温度場です。§7 の u は変位です。既存の式を追う際は、温度の u をノートでは T と書き換えて構

いません。温度と変位を同じ物理量として扱わないでください。

記号	大きさ・単位	意味
N, m, r	20,696, 121, 5	セル数、保存時刻数、採用モード数
S, X	$N \times m, K$	温度履歴、各セルの時間平均を引いた履歴
$\bar{u}$	$N \times 1, K$	各セル固有の時間平均温度
U, Σ, V^T	薄い SVD で $N \times m, m \times m, m \times m$	左特異ベクトル、特異値、右特異ベクトルの転置
$\Phi = U_r$	$N \times r$ 、無次元	上位 r 本の空間モード
$a(t)$	$r \times 1, K$	時刻 t で全セルが共有するモード係数
P	$5 \times N$ 、無次元	選んだセルの行を取り出す選択行列

T は転置、 $+$ は Moore–Penrose 疑似逆行列、 σ_k は特異値、 λ_k は固有値です。本文の「 r 点 P 」は選んだ点集合を指します。行列として書くと $U_{r,P} = P\Phi$ です。POD の「エネルギー」は温度変動の二乗和であり、熱量 [J] ではありません。この実装はセル体積の重みを付けない SVD です。

平均場と基底は 0~600 秒の 121 枚から作って固定します。300 秒でヒータを切っても平均場だけを切り替えません。

2. POD の数式 — なぜ SVD で場の「型」が出るのか

2-1. スナップショット行列 — 温度場の「写真」を横に並べる

まず言葉で。ある時刻の温度場（全セルの温度）を 1 枚の写真だと思ってください。シミュレーションは時刻 $t_1, t_2, \dots, t_m$ の写真を m 枚くれます。この写真を **1 枚 = 1 列** にして横に並べたのが、スナップショット行列 S です（行 = 場所 = セル、列 = 時刻。 $u_i(t_k)$ = セル i の時刻 t_k の温度）：

$$S = \begin{pmatrix} u_1(t_1) & u_1(t_2) & \cdots & u_1(t_m) \\ u_2(t_1) & u_2(t_2) & \cdots & u_2(t_m) \\ \vdots & \vdots & \ddots & \vdots \\ u_N(t_1) & u_N(t_2) & \cdots & u_N(t_m) \end{pmatrix}$$

（横に読むと 各行が **1 セル**の時間履歴、縦に読むと各列が **1 時刻**の温度場（**1 枚**の写真））

- 縦の **1 列** $u(t_k) = (u_1(t_k), \dots, u_N(t_k))^T \dots$ その時刻の温度場まるごと（**1 枚**の写真）。
- 横の **1 行** ... あるセルの温度が時間とともにどう動いたか（**1 点**の時間履歴）。
- 本ケースは $N = 20,696$ セル、 $m = 121$ 枚（0~600 秒）。

次に各セルから「そのセル自身の時間平均」を引きま

す。平均は下地なので、そこからの変動だけを見たいからです。セル i の時間平均を $\bar{u}_i = \frac{1}{m} \sum_k u_i(t_k)$ とし、これを各行から引いた行列を X と書きます：

$$X = [u(t_1) - \bar{u}, \dots, u(t_m) - \bar{u}], \quad X_{ik} = u_i(t_k) - \bar{u}_i.$$

- $\bar{u} = (\bar{u}_1, \dots, \bar{u}_N)^T \dots$ セルごとの時間平均（1 枚の「平均写真」）。
- X の各成分は「そのセルが、その時刻に、自分の平均からどれだけ上下したか（変動）」。
- 各行の平均はちょうど 0（下地を抜いたので、残るのは揺れだけ）。

この変動だけの写真束 X から共通の「型」を取り出すのが、次の POD です。

2-2. 「最良の型」を求める変分問題 → 固有値問題

場を 1 つの単位ベクトル φ ($\|\varphi\| = 1$) で最もよく表す、とは各スナップショットの φ 方向成分の二乗和を最大化すること：

$$\max_{\|\varphi\|=1} \sum_{k=1}^m |\varphi^T(u(t_k) - \bar{u})|^2 = \max_{\|\varphi\|=1} \varphi^T(XX^T)\varphi.$$

制約 $\varphi^T\varphi = 1$ をラグランジュ乗数 λ で組み込みます。

$$\mathcal{L} = \varphi^TXX^T\varphi - \lambda(\varphi^T\varphi - 1).$$

対称行列 A に対して $\partial(\varphi^TA\varphi)/\partial\varphi = 2A\varphi$ なので、

$$\frac{\partial \mathcal{L}}{\partial \varphi} = 2XX^T\varphi - 2\lambda\varphi = 0.$$

両辺を 2 で割って移項すると、次の固有値問題になります：

$$(XX^T)\varphi = \lambda\varphi$$

- 固有ベクトル $\varphi_k = \mathbf{POD}$ モード（場の型）
- 固有値 $\lambda_k = \sigma_k^2 =$ その型のエネルギー（大きいほど支配的）

2-3. SVD との同値（実際の計算法）

X を特異値分解 $X = U\Sigma V^T$ すると

$$XX^T = U\Sigma V^T V \Sigma U^T = U\Sigma^2 U^T.$$

つまり U の各列がそのまま XX^T の固有ベクトル = **POD モード**、 σ_k^2 が固有値。巨大な XX^T ($N \times N$) を作らず、**SVD** 一発で **POD** が得られます。しかも上位 r モードで打ち切った近似は、ランク r の全行列の中でフロベニウス誤差を最小にする (Eckart-Young の定理) = 「POD は最もエネルギー効率のよいモード分解」です。

■実際のコードは 3 行 — np.linalg.svd の中で何が起きているか ここまでが理屈ですが、実装は拍子抜けするほど短いです。本研究のモード分解は run/select_points_qdeim.py の次の 3 行がすべてです (専用ライブラリは使っていません)。

```
mean = X.mean(axis=1)          # セルごとの時間
                                ↳ 平均
Xc = X - mean[:, None]        # 平均を引いて「変動
                                ↳ だけ」にする
U, S, Vt = np.linalg.svd(Xc, full_matrices=False) # ← これが
                                ↳ モード分解
```

出てくる 3 つの箱の意味は次のとおりです (本ケースの実サイズつき)。

変数	数学記号	中身	サイズ
Xc	X	平均を引いた温度場 (1 列 = 1 時刻)	20,696 × 121
U	U	POD モード =空間の型 (1 列 = 1 モード)	20,696 × 121
S	σ_k	各モードの大きさ (特異値)	121
Vt	V^T	各モードの時間の動き方	121 × 121

full_matrices=False は「20,696 × 20,696 の巨大な U を作らず、意味のある 121 列だけ計算する」という指定です。

中でやっていることを一言でいうと、行列を次のように足し算へ分解しています:

$$X = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T$$

各項が「空間の型 u_k × 時間の動き v_k × 大きさ σ_k 」の 1 セット。

行列の形で書くようになっていきます (N = セル数、 m = 時刻数、 r = モード数) :

$$\underset{N \times m}{X} = \underset{N \times r}{U} \underset{r \times r}{S} \underset{r \times m}{V^T} = \begin{pmatrix} | & | & \cdots & | \\ u_1 & u_2 & & u_r \\ | & | & & | \end{pmatrix} \begin{pmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_r \end{pmatrix} \begin{pmatrix} -v_1^T - \\ -v_2^T - \\ \vdots \\ -v_r^T - \end{pmatrix}$$

- u_k は縦ベクトル (長さ N) = 全セルぶんの重み = **1 枚の温度分布の絵**
- v_k は縦ベクトル (長さ m) = 全時刻ぶんの係数 = **その絵の濃さの時間変化**
- $\sigma_k u_k v_k^T$ は縦 × 横の外積なので、 X と同じ $N \times m$ の行列になる

しかも $\sigma_1 \geq \sigma_2 \geq \cdots$ と大きい順に並ぶので、頭からいくつか取れば良い近似になります。(アルゴリズムとしては LAPACK の gesdd が呼ばれ、二重対角化してから反復で特異値を求めます。ただし使う側が知っておくべきは上の分解の意味だけです。)

■数字で最後まで追う — 3 点 × 4 時刻の完全分解 言葉だけでは分かりにくいので、実データから **3 点 × 4 時刻** だけ抜き出して全部書き下します (P2 = ヒータ側、P0 = 中央高さ、P4 = 反対側の底面。時刻は 50, 150, 300, 500 秒)。

入力 (温度、単位は °C) と、各行から時間平均 m を引いた X :

$$\begin{pmatrix} 22.182 & 23.867 & 25.742 & 23.664 \\ 21.615 & 23.307 & 25.239 & 23.667 \\ 20.097 & 20.922 & 22.648 & 23.573 \end{pmatrix} - \begin{pmatrix} 23.864 \\ 23.457 \\ 21.810 \end{pmatrix} = \underbrace{\begin{pmatrix} -1.682 & 0.003 & 1.879 & -0.199 \\ -1.842 & -0.150 & 1.782 & 0.210 \\ -1.712 & -0.888 & 0.838 & 1.763 \end{pmatrix}}_{X \ (3 \times 4)}$$

これを np.linalg.svd かけると、次の 3 つが返ります:

$$U = \begin{pmatrix} 0.566 & 0.513 & 0.646 \\ 0.608 & 0.269 & -0.747 \\ 0.557 & -0.815 & 0.159 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 4.161 & & \\ & 1.802 & \\ & & 0.036 \end{pmatrix}, \quad V^T = \begin{pmatrix} -0.727 & -0.140 & 0.628 & 0.240 \\ 0.020 & 0.380 & 0.422 & -0.823 \\ 0.470 & -0.765 & 0.421 & -0.126 \end{pmatrix}$$

ここからが本題です。各項 $\sigma_k u_k v_k^T$ を実際に計算すると、どれも X と同じ 3×4 の行列になります:

第 1 項 ($\sigma_1 = 4.161$ 、エネルギー 84.20 %) — u_1 は 3 成分とも同符号・同程度なので「全体が一緒に上下する型」:

$$4.161 \begin{pmatrix} 0.566 \\ 0.608 \\ 0.557 \end{pmatrix} \begin{pmatrix} -0.727 & -0.140 & 0.628 & 0.240 \end{pmatrix} = \begin{pmatrix} -1.712 & -0.331 & 1.478 & 0.564 \\ -1.839 & -0.355 & 1.589 & 0.606 \\ -1.685 & -0.326 & 1.456 & 0.555 \end{pmatrix}$$

第 2 項 ($\sigma_2 = 1.802$ 、15.79 %) — u_2 は上 2 点が正・底面が負なので「上下の偏りの型」:

$$1.802 \begin{pmatrix} 0.513 \\ 0.269 \\ -0.815 \end{pmatrix} \begin{pmatrix} 0.020 & 0.380 & 0.422 & -0.823 \end{pmatrix} = \begin{pmatrix} 0.019 & 0.351 & 0.390 & -0.760 \\ 0.010 & 0.184 & 0.205 & -0.399 \\ -0.030 & -0.558 & -0.620 & 1.208 \end{pmatrix}$$

第 3 項 ($\sigma_3 = 0.036$ 、0.01 %) — 成分がすべて 0.02 以下の「ほぼ何もない項」:

$$0.036 \begin{pmatrix} 0.646 \\ -0.747 \\ 0.159 \end{pmatrix} \begin{pmatrix} 0.470 & -0.765 & 0.421 & -0.126 \end{pmatrix} = \begin{pmatrix} 0.011 & -0.018 & 0.010 & -0.003 \\ -0.013 & 0.020 & -0.011 & 0.003 \\ 0.003 & -0.004 & 0.002 & -0.001 \end{pmatrix}$$

3 項を足すと元に戻ります (差の最大値 1.1×10^{-15}) :

$$\begin{pmatrix} -1.712 & -0.331 & 1.478 & 0.564 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} + \begin{pmatrix} 0.019 & 0.351 & 0.390 & -0.760 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} + \begin{pmatrix} 0.011 & -0.018 & 0.010 & -0.003 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} = \begin{pmatrix} -1.682 & 0.003 & 1.879 & -0.199 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} = X$$

打ち切ったときの誤差は:

使う項	エネルギー	最大誤差
第 1 項だけ	84.20 %	1.208 °C
第 2 項まで	99.99 %	0.020 °C
第 3 項まで	100 %	0 (厳密)

→第 3 項を捨てても **0.02 °C** しか変わらない。これが「少ないモードで済む」の正体です。20,696 セル版でも構造は同じで、項が 121 個あるうち **2 項で 99.99 %** になります。

本当にそうになっているか、実データで確かめます。検証スクリプトを同梱しました:

OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4

↪ python3 run/svd_explained.py

実行結果 (20,696 セル × 121 時刻、0~600 秒):

確かめたこと	結果
① $X = U \Sigma V^T$ に分解できているか	相対誤差 1.2×10^{-15} = 厳密に分解 (近似ではない)
② U の列は XX^T の固有ベクトルか	$XX^T u_i$ と $\sigma_i^2 u_i$ の差 4.3×10^{-16} = 一致
③ 時間側の小行列 (121×121) でも同じか	特異値 1822.0, 540.7, 61.7, 12.0 が完全一致 (スナップショット法)
④ モードは正規直交か	$U^T U = I$ (単位行列)

②は §2-2 の固有値問題と、③は「巨大な XX^T を作らなくてよい」という主張の実証です。

何モードで足りるか (④ Eckart–Young の実測):

モード数	場の相対誤差	温度の RMSE
1	28.62 %	0.344 K
2	3.31 %	0.040 K
3	0.64 %	0.008 K
5	0.03 %	0.0004 K

ここで注意が **1** つあります。「2 モードでエネルギー 99.99 %」と「相対誤差 3.31 %」は矛盾しません。エネルギーは **2 乗の量** なので、誤差は $\sqrt{1 - 0.9999} = 0.033 = 3.3 \%$ になります。エネルギー比は実際の誤差よりも良く見えるので、モード数を決めるときは温度の RMSE (ここでは 0.04 K) のような物理量での誤差も併せて見るのが安全です。

最後に V_t (時間の動き) の中身も見ておきます。 $\sigma_k v_k$ が「モード k が各時刻にどれだけ効いているか」です:

時刻 [s]	0	50	150	300	450	600
モード 1 の係数	401.9	301.0	117.1	- 146.7	- 120.3	- 110.0
モード 2 の係数	114.3	15.0	- 44.3	- 83.6	42.8	51.6

- モード 1 は加熱の進行とともに単調に変化＝「全体の温度レベル」を担当
- モード 2 は 300 秒 (ヒータ OFF) の前後で符号が反転＝「左右の偏りの向き」を担当 (加熱中はヒータ側が熱い状態、冷却後はその偏りが解消・逆転していく様子)

つまり np.linalg.svd は、**121 枚** の温度場から「全体の昇温」と

「左右の偏り」という **2 つ** の型を自動で見つけ出し、それぞれの時間変化まで分離してくれたわけです。物理の知識を一切与えずに、です。

(符号には任意性があります。 u_k と v_k の両方を同時に反転しても同じ X になるため、モードの色の向きが逆に出ることがあります。物理的な意味は変わりません。)

■自分のケースで試す —CFD 結果から POD までの手順 (8 ステップ) 「行列を入れれば出る」とはいえ、**CFD** の結果をその行列にするまでが実務では一番手間です。本研究で実際に踏んだ手順を、そのまま再現できる形で書きます。

STEP 1. CFD を回し、スナップショットを等間隔で残す

system/controlDict で出力を設定します (本研究の実際の値):

```
application chtMultiRegionFoam;
endTime 600;
writeControl adjustableRunTime;
writeInterval 5; // 5 秒ごとに出力
writeFormat ascii; // 後で読むので ascii が楽
```

→ 0~600 秒で **121 枚** のスナップショットができます。ここで
の注意は 3 つ:

- 等間隔で出す: POD は各スナップショットを等しい重みで扱うため、間隔が不揃いだとか細かく出した時間帯が実質的に重くなります
- **ascii** 形式: binary でも読めますがパーサが面倒になります (容量と引き換え)
- 並列計算なら reconstructPar で 1 つに戻してから読む

STEP 2. セル中心座標 (と体積) を書き出す

温度の値だけでは「どのセルがどこにあるか」が分かりません。後で図示・選点に使うので出しておきます:

```
postProcess -region solid -func writeCellCentres -time 5
postProcess -region solid -func writeCellVolumes -time 5 # 体
↪ 積重みを使う場合
```

→ 5/solid/C (セル中心座標)、5/solid/V (セル体積) が生成されます。

STEP 3. 温度場ファイルを読んで行列に積む

OpenFOAM のフィールドファイルは internalField nonuniform List<scalar> 20696 (...) という形式です。正規表現で中身を取り出します (本研究の read_foam_field() の要点):

```
def read_foam_field(path, ncell=None):
    txt = open(path).read()
    seg = txt[txt.find("internalField");:]
    if "uniform" in seg[:40] and "nonuniform" not in seg[:40]: #
        ↪ 均一場 (初期条件など)
        v = float(re.findall(r"uniform\s+([-d.eE ]+);", seg)[0])
        return np.full(ncell, v)
    n = int(re.search(r"List<\w+>\s*\n?\s*(\d+)", seg).group(1))
    body = seg[seg.find("(")+1:]
```

```
return np.array([float(x) for x in re.findall(r"[-+]?[d]\d\.eE+-+",
↪ body)[:n]])
```

```
times = sorted(int(d) for d in os.listdir(case) if d.isdigit())
X = np.column_stack([read_foam_field(r"{case}/{t}/solid/T",
↪ ncell) for t in times])
```

ここが一番の落とし穴です:

- 初期時刻は uniform で書かれることがある（全セル同じ値なので行数がない）→上のように分岐が要る
- 全時刻でセルの並び順が同じである必要がある（同じメッシュなら自動的に満たされる。ただし動的メッシュやリメッシュを使うと崩れるので POD 不可）
- 0.orig のような数字でないディレクトリを除外する

STEP 4. 平均を引く

```
mean = X.mean(axis=1) # セルごとの時間平均
Xc = X - mean[:, None]
```

引かないとモード 1 が「平均場そのもの」になり、変動の型が 2 番目以降にずれます。

STEP 5. SVD をかける

```
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
```

full_matrices=False を必ず付けます。付けないと $N \times N$ (20,696 角) の行列を作ろうとしてメモリが破綻します。

STEP 6. エネルギーを見てモード数を決める

```
energy = S**2 / (S**2).sum()
print(np.cumsum(energy)[:5]) # → 0.918, 0.999, ...
```

ここで §2-3 で述べた「エネルギーは 2 乗の量」の注意が効きます。99.9 % でも誤差は 3.3 % なので、物理量（温度 RMSE）でも確認してから打ち切ります:

```
for r in [1, 2, 3, 5]:
    Xr = (U[:, :r] * S[:r]) @ Vt[:r]
    print(r, np.sqrt(((Xc - Xr)**2).mean()), "K")
```

STEP 7. モードを可視化して物理的に意味を確認する

数値だけで進めず、必ず絵にして確認します。 U の各列は「全セル分の値」なので、そのまま CFD のスカラー場として書き出せば ParaView で見られます。モード 1 が「全体の昇温」、モード 2 が「左右の偏り」のように物理的に読めるかをチェックします。読めない場合は、データの取り方（時間窓・出力間隔）を疑います。

STEP 8. 代表点を選ぶ (Q-DEIM)

```
from scipy.linalg import qr
_, _, piv = qr(U[:, :r].T, pivoting=True) # 枢軸 QR
points = piv[:r] # 先頭 r 個が代表点
```

詳細は §4 で説明します。

まとめ（つまづきやすい点）

ステップ	よくある失敗	対策
1	出力間隔が不揃い	adjustableRunTime で等間隔に
1	並列計算のまま読む	reconstructPar してから
3	初期時刻が uniform で落ちる	分岐して全セル同値で埋める
3	動的メッシュでセル順が変わる	POD には使えない（固定メッシュ前提）
5	メモリ不足	full_matrices=False
6	エネルギーだけで打ち切る	物理量の誤差でも確認
7	絵にせず数値だけで進む	ParaView でモードを必ず目視

■この方法の限界 一発表や査読で必ず聞かれる 2 点 ここまで「行列を入れればモードが出る」と書きましたが、手軽さの裏返し of 限界が 2 つあります。正直に書いておきます。

限界 1: 出てくるのは「入れたデータの型」であって、物理が決めた固有モードではない

POD モードは形状・材料・境界条件だけでは決まりません。どんな運転をしたか（加熱位置・出力・時間窓）まで含めて決まります。対比すると分かりやすいはず:

	固有モード（熱伝導方程式の固有関数）	POD モード（本記事）
何で決まるか	形状・材料・境界条件だけ	実際に起きた温度場データ
熱源の影響	受けない	受ける
意味	その物体が持つ固有の減衰形状	「その運転で実際によく現れた型」

本ケースで モード 1 = 全体の昇温、モード 2 = 左右の偏りという素直な形が出たのは、熱伝導が線形で熱源が 1 つ（ON/OFF のみ）と単純だからです。ヒータ位置を変えればモード 2 の形は変わります。この訓練条件依存は §8-3 の未解決問題そのものです。

限界 2: セル体積の重みを付けていない

上のコードは各行（セル）を等しく扱っています。しかし本来の POD（連続体のエネルギー最大化）は、体積重み $\sqrt{V_i}$ を掛けた行列

$$\tilde{X}_{ik} = \sqrt{V_i} X_{ik}$$

を分解するのが正しい定義です。等重みだと細かいセルが密集した領域が実質的に重くなります。

「本ケースのメッシュはそこまで不均一ではないだろう」と考えていたのですが、実際に測ったら違いました。OpenFOAM でセル体積を書き出して確認すると:

- セル体積は $1.2 \times 10^{-9} \sim 2.3 \times 10^{-7} \text{ m}^3$ = 最大と最小で 192 倍の開き（固体の合計 317 cm^3 ）

そこで、体積重みを付けた POD と等重み POD を比較しました (run/pod_volume_weight_check.py):

比較項目	等重み（従来）	体積重み（厳密）
累積エネルギー（1 モード）	91.81 %	93.69 %
累積エネルギー（2 モード）	99.89 %	99.93 %
Q-DEIM で選ばれた 5 点	7634, 7924, 7765, 859, 733	12920, 7634, 7924, 859, 838
5 点復元の体積加重 RMSE	0.00072 K	0.00067 K

モードの向きの一致度（体積内積で規格化）はモード **1** が **0.9998**（ほぼ同一）、モード **2** が 0.966、モード **3** が 0.922 でした。つまり：

- 支配的なモード **1** は事実上変わらない — 全体の昇温という型は重み付けに鈍感
- 高次モードほど差が出る — 細かい構造ほど重み付けの影響を受ける
- 選点は **5** 点中 **2** 点が入れ替わった — 「どこを測るか」の答えは重み付けで変わりうる
- ただし復元精度はどちらも **0.0007 K** 程度で、観測ノイズ（0.3 K）よりはるかに小さい

結論：本ケースでは実害は小さい（復元精度に有意差なし）ものの、厳密には体積重み付きが正しく、選点が変わる可能性があるという点は、条件が変わったときのために認識しておくべき限界です。

2-4. 用語の整理 — POD・SVD・特異値分解の違い

ここで混乱しやすい 3 語を整理します。まず大前提：

SVD = 特異値分解（Singular Value Decomposition）は同じもの。英語の略称か日本語かの違いだけで、中身は 1 つ。だから区別すべきは実質「**SVD**（＝特異値分解）」と「**POD**」の 2 つです。

- **SVD**（＝特異値分解）… どんな行列にも使える線形代数の”道具”。行列を $X = U\Sigma V^T$ の 3 つに分解する操作、それ以上でも以下でもありません。物理も温度も知りません。
- **POD**（固有直交分解）… データから”エネルギー最大の型”を取り出す”目的・考え方”。その計算に SVD を使います。「POD をする」＝実際には「平均を引いた X を SVD し、 U の列を”モード”、 σ_k^2 を”エネルギー”と読む”こと。

用語	正体	立場
特異値分解	SVD の日本語名（＝ SVD そのもの）	—
SVD	$X = U\Sigma V^T$ の行列分解	汎用の 計算道具
POD	場のエネルギー最大の直交モードを取る	目的・解釈（平均除去・重み・エネルギー意味づけを追加）

つまり「**POD** は **SVD** と別物」ではなく、**SVD** は **POD** を計算する道具。POD 側には「①平均を引く②（体積などで重みを付けた）エネルギーで最良性を測る③上位数モードで場を代表させる」という”味付け”が加わります。統計の **PCA**（主成分分析）や確率過程の **Karhunen–Loève 展開**も、呼び名が違っただけで本質は同じ仲間です。

（別解：SVD を使わず XX^T の固有値分解や、時間側の小さい $X^T X$ を解く”スナップショット法”でも、同じ POD モードが得られます。SVD はその中で最も安定・簡単な計算法という位置づけ。）

2-5. コラム：POD 以外のモデル縮約 — クリロフ部分空間

法との違い

モデル縮約（MOR: Model Order Reduction）には POD 以外の流派もあります。代表はクリロフ部分空間法（モーメントマッチング、Arnoldi 法など）と平衡打ち切り（balanced truncation）。どれも「大きい系を小さい部分空間へ射影する」仲間で、違いは部分空間の作り方です：

	POD	クリロフ部分空間法	平衡打ち切り
部分空間の材料	データ（スナップショットの SVD）	方程式の行列： $\text{span}\{b, Ab, A^2b, \dots\}$	可制御性・可観測性グラミアン
事前に必要なもの	シミュレーション結果だけ	システム行列 A, b （FEM の C, K ）	システム行列＋リアプノフ方程式求解
何に忠実か	訓練データのエネルギー	伝達関数のモーメント（周波数応答）	入出力の重要度（誤差上界つき）
非線形への拡張	DEIM 等で可能	基本は線形（LTI）系向け	基本は線形系向け
弱点	訓練条件に依存（外挿に弱い）	侵入的＝行列へのアクセスが必要	大規模系では計算が重い

■そもそも「クリロフ部分空間」とは何か — 3 ノードの手計算で 名前は難しいですが、定義は 1 行です。行列 A と入力ベクトル b から作る

$$\mathcal{K}_r(A, b) = \text{span}\{b, Ab, A^2b, \dots, A^{r-1}b\}$$

＝「 b に A を繰り返して掛けてできる方向を集めた空間」。これだけです。物理的な意味は「入力の影響が系の中を伝わっていく順番」。3 ノードの熱伝導（ヒータは端の節点 1）で確かめます：

$$A = \begin{pmatrix} -1 & 1 & 0 \\ 1 & -2 & 1 \\ 0 & 1 & -1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad (\text{node 1 heated}).$$

A を順に掛けると：

$$b = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad Ab = \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad A^2b = \begin{pmatrix} 2 \\ -3 \\ 1 \end{pmatrix}.$$

- b ：熱はまだ 節点 **1** だけ
- Ab ：節点 **2** に届いた（成分が非ゼロに）
- A^2b ：節点 **3** まで届いた

つまり A を 1 回掛ける＝熱が隣へ 1 ステップ伝わる。クリロフ基底は「ヒータの影響が届く順」に空間を張っていきます。これが解と直結する理由は、熱方程式 $\dot{T} = AT + bQ$ の解に出てくる行列指数関数のテイラー級数

$$e^{At}b = b + tAb + \frac{t^2}{2}A^2b + \dots$$

がまさにクリロフ方向の和だから。早い項ほど支配的なので、最初の r 方向だけ残して打ち切っても良い近似になる＝これがクリロフ縮約の原理です。実際の手順は：

1. $b, Ab, A^2b, \dots$ を **Arnoldi 法**（グラム・シュミット直交化と同じ発想）で直交基底 V_r にする（生の $A^k b$ はすぐ同じ向きに揃ってしまい数値的に危ないため）。
2. 大きい系を $T \approx V_r a$ と置いて射影： $\dot{a} = (V_r^T A V_r) a + V_r^T b Q$ 。これで N 次元の方程式が $r \times r$ の小さい方程式になります。

POD との対比で言うと：**POD** は「シミュレーション結果（データ）」から支配方向を後付けで学ぶのに対し、クリロフは「方程式（ A, b ）」から支配方向を計算前に組み立てる。だからクリロフはスナップショット不要ですが、行列 A, b に触れる必要がある（侵入的）、という表の違いにつながります。

本研究が **POD** を選んだ理由は 2 つ：

1. 非侵入的（**non-intrusive**）だから。クリロフ法や平衡打ち切りは A, b という行列そのものが必要ですが、OpenFOAM の CHT ソルバから系行列を取り出すのは大変です。POD は温度スナップショット（**solid/T** のファイル）さえあればいい。
2. 「代表点」が欲しかったから。POD → Q-DEIM は物理的な点（セル）を選んでもくれるので、センサ議論・集中定数 ROM・観測設計に直結します。クリロフ法の縮約座標は抽象的な部分空間で、「どの場所」という解釈が付きません。

なお熱伝導は線形なので、行列が手に入る場合（FrontISTR 側など）はクリロフ法も有力な代替です（工作機械の熱モデル縮約で実績があります）。また本記事の ROM は厳密には「POD の Galerkin 射影 ROM」ではなく、**Q-DEIM** 点上の集中定数モデルを校正したもの（POD は点選ぴと場の復元に使用）。 C_i, K_{ij}, h という物理的に読める係数にしたことで、EnKF の拡大状態（ Q, h 推定）へ自然につながります。

3. 小さな行列で POD を手計算する（3 セル × 4 時刻・文字式）

言葉だけでは腑に落ちないので、**3 セル (A,B,C) × 4 時刻** の最小例で最後まで計算します。数字だと構造が見えにくいので、文字式で進めます。ポイントは、各セルの温度を

$$u_i(t_k) = \bar{u}_i + a_i g_k, \quad \sum_{k=1}^4 g_k = 0$$

と置くこと。ここで $\bar{u}_i$ は下地（時間平均）、 a_i はその場所の変動の強さ、 g_k は全セル共通の時間プロファイル（平均を引いてあるので総和ゼロ）。「場所ごとの強さ a_i 」と「時間の動き g_k 」の積で温度が決まる、という素直な仮定です。

イメージはこうです——中空円筒を片側から加熱し、ヒータ側から順に A（強）・B（中）・C（弱）の 3 セルを取ります。3 セルとも同じように昇温しますが（共通の g_k ）、ヒータに近いほど大きく振れます（ $a_A > a_B > a_C$ ）：

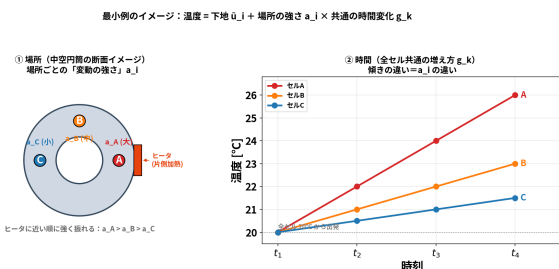


図 1: 最小例のイメージ（中空円筒断面と温度の時間変化）

左＝場所（断面イメージ）：ヒータに近い順に「変動の強さ」 a_i が大きい。右＝時間：3 セルは同じ増え方 g_k を共有し、傾き（振れ幅）の違いが a_i の違い。この「場所 a_i × 時間 g_k 」という掛け算の形が、次で効いてきます。

セル	t_1	t_2	t_3	t_4	時間平均
A（ヒータ側）	$\bar{u}_A + a_A g_1$	$\bar{u}_A + a_A g_2$	$\bar{u}_A + a_A g_3$	$\bar{u}_A + a_A g_4$	$\bar{u}_A$
B（中間）	$\bar{u}_B + a_B g_1$	$\bar{u}_B + a_B g_2$	$\bar{u}_B + a_B g_3$	$\bar{u}_B + a_B g_4$	$\bar{u}_B$
C（反対側）	$\bar{u}_C + a_C g_1$	$\bar{u}_C + a_C g_2$	$\bar{u}_C + a_C g_3$	$\bar{u}_C + a_C g_4$	$\bar{u}_C$

ヒータ側ほど強く反応するので $a_A > a_B > a_C > 0$ 。各セルから自分の平均 $\bar{u}_i$ を引くと、POD に渡す変動行列 X は 場所ベクトル a と 時間ベクトル g の外積 になります：

$$X = \begin{pmatrix} a_A g_1 & a_A g_2 & a_A g_3 & a_A g_4 \\ a_B g_1 & a_B g_2 & a_B g_3 & a_B g_4 \\ a_C g_1 & a_C g_2 & a_C g_3 & a_C g_4 \end{pmatrix} = \begin{pmatrix} a_A \\ a_B \\ a_C \end{pmatrix} \begin{pmatrix} g_1 & g_2 & g_3 & g_4 \end{pmatrix} = a g^T.$$

この時点で $X = a g^T$ = ランク 1（1 つの空間パターン × 1 つの時間プロファイル）。「場の変動は、たった 1 つの型で説明できる」ことが、もう形から見えています。

3-1. 相関行列 XX^T を作る

$X = a g^T$ を代入するだけです。 $g^T g = \|g\|^2$ はただのスカラーなので、

$$XX^T = a g^T g a^T = \|g\|^2 a a^T, \quad \|g\|^2 = \sum_{k=1}^4 g_k^2.$$

右辺は 場所ベクトル a の外積 aa^T の定数倍 = やはり ランク 1。時間プロファイル g は「大きさ $\|g\|^2$ 」という 1 個の数にまとも、空間の型は a だけで決まることが読み取れます。

3-2. 固有値と POD モード（文字式）

ランク 1 行列 $XX^T = \|g\|^2 aa^T$ の非ゼロ固有ベクトルは a の向き、固有値はその係数です。POD モード φ_1 とエネルギー λ_1 は

$$\varphi_1 = \frac{a}{\|a\|}, \quad \lambda_1 = \sigma_1^2 = \|g\|^2 \|a\|^2.$$

他の固有値はすべて 0 なので、エネルギー比は

$$\frac{\lambda_1}{\text{tr}(XX^T)} = \frac{\|g\|^2 \|a\|^2}{\|g\|^2 \|a\|^2} = 1,$$

すなわち モード 1 だけで **100 %**。モード $\varphi_1 = a/\|a\|$ の中は「各セルの変動の強さの比 $a_A : a_B : a_C$ 」そのもの。つまり POD モードとは「どの場所がどれだけ強く動くか」という空間パターンで、 $\bar{u}$ = 下地、 φ_1 = そこからの変動の型、という役割分担です。

（本物の場は完全なランク 1 ではありませんが、後で見るようにモード 1 で **91.8 %** まで説明でき、構図は同じです。）

4. Q-DEIM ーどの点を測れば場を復元できるか

Q-DEIM は「少数のセンサで場全体を復元するのに、どこにセンサを置けばいいか」を決める方法です。名前は難しいですが、やっていることは直感的なので、順を追ってほぐします。

4-0. その前に ー Q-DEIM と gappy-POD は「別の道具」です

この 2 つは名前が似ていて必ずセットで出てくるので混同しやすいのですが、やっていることも、実行するタイミングも違います。先にここをほぐしておきます。

■一言でいうと

	Q-DEIM	gappy-POD
ひとことで	どこを測るか決める	測った値から全部を埋める
いつやるか	オフラインで 1 回だけ（設計時）	オンラインで毎時刻（運用時）
入力	POD モード行列 U_r (20,696×5)	選んだ 5 点の測定値 T_p (5×1)
出力	測る点の番号 $P = \{P_1, \dots, P_5\}$	全セルの温度 $\hat{T}$ (20,696×1)
中身	枢軸付き QR 分解（食欲法）	5 元連立を解く + 行列ベクトル積
計算量	1 回きり。数秒	毎回。5 ³ + 20696 × 5 程度＝一瞬
例えると	センサの設置工事	毎日のメーター読み

Q-DEIM がセンサの置き場所を決め、**gappy-POD** がそのセンサの値を読んで全体像を描く。工事は最初の 1 回、読み取りは毎時刻 ―― これが役割分担です。

■名前の由来

- **gappy**（ギャッピー）は英語の *gap*（すき間・穴）から来ていて、「穴だらけ」という意味です。20,696 セルのうち値が分かっているのはたった 5 セル。残り 20,691 セルは空欄。この歯抜けのデータから元の絵を復元するので **gappy-POD** と呼ばれます。もともとは欠損のある画像を補完する手法として提案されたもので、「虫食い算を解く」イメージが近いです。
- **Q-DEIM** は QR 分解を使う **Discrete Empirical Interpolation Method** の略。DEIM が「離散的な点で補間するために点を選ぶ方法」で、その選び方を **QR** 分解でやるのが Q-DEIM です。頭の Q は、まさに `scipy.linalg.qr(..., pivoting=True)` の Q です。

■なぜ「穴だらけ」でも復元できるのか ふつうに考えれば、20,696 個の数を 5 個の測定値から復元するのは無理です。情報が足りません。できるのは、**POD** が先に「答えの形」を教えてくれているからです。

§3 で見たとおり、温度場は

$$\hat{T} = \bar{T} + a_1 \varphi_1 + a_2 \varphi_2 + \dots + a_5 \varphi_5$$

と書けました。 $\bar{T}$ も φ_k も計算済みで固定。時刻ごとに変わるのは $a_1, \dots, a_5$ の 5 個だけです。

つまり「20,696 個を当てる問題」は、**POD** を通した瞬間に「5 個を当てる問題」に化けています。5 個の未知を決めるのに必要な式は 5 本。だから 5 点測れば足りる ――これが種明かしです。

穴だらけでいいのは「穴を 1 つずつ埋める」のではなく、全部まとめて 5 個のつまみで決めているからです。5 個のつまみを合わせれば、測っていないセルも自動的に決まります。

■流れを図にすると



上半分（POD → Q-DEIM）は設計時に 1 回。下半分（測る → gappy-POD）は毎時刻くるくる回るループです。

■gappy-POD の中身（3 ステップ）

手順	やること	式	サイズ
①	5 点を測り、平均からのズレにする	$\delta_P = T_P - \bar{T}_P$	5×1
②	5 元連立を解いてつまみ a を出す	$a = (U_{r,P})^{-1} \delta_P$	(5×5) × ¹ ×(5×1) → 5×1
③	そのつまみを全セルに使い回す	$\hat{T} = \bar{T} + U_r a$	(20,696×5)×(5×1) → 20,696×1

$U_{r,P}$ は U_r (20,696×5) から「測った 5 行」だけを抜き出した 5×5 行列です。②で逆行列を取るのはこの 5×5 だけ。だから毎時刻回しても一瞬で終わります。

②を展開すると、ふつうの連立方程式であることがはっきりします：

$$\begin{aligned} \delta_{P_1} &= a_1 \varphi_1[P_1] + a_2 \varphi_2[P_1] + \dots + a_5 \varphi_5[P_1] &< 1 \text{ 点目で式 1 本} \\ \delta_{P_2} &= a_1 \varphi_1[P_2] + a_2 \varphi_2[P_2] + \dots + a_5 \varphi_5[P_2] &< 2 \text{ 点目で式 1 本} \\ &\vdots \\ \delta_{P_5} &= a_1 \varphi_1[P_5] + \dots + a_5 \varphi_5[P_5] &< 5 \text{ 点目で式 1 本} \end{aligned}$$

1 点測るごとに式が 1 本立つ。未知が 5 個なので、5 点でちょうど解ける形になります。

③をセル c 1 個について書くと：

$$\hat{T}_c = \bar{T}_c + a_1 \varphi_1[c] + a_2 \varphi_2[c] + \dots + a_5 \varphi_5[c]$$

$\varphi_k[c]$ はセル c が測定点でなくても持っている値（モードは全セル分そろっている）。だから測っていないセルも、モードを通して値が決まるわけです。

■そこで Q-DEIM の出番 ー「どの 5 行を抜くか」で連立の解きやすさが変わる ここまで来ると、Q-DEIM の役割がはっきりします。②の連立を解くのは $U_{r,P}$ = モード行列からの 5 行を抜いたかで決まる行列です。

どの行を抜くか = どこに測定点を置くか。つまり測定点選びは「5×5 の連立方程式を解きやすくする問題」そのものになっています。

行がそっくりな点を選んでしまうと「同じ式を何回も書いている」状態になり、連立が解けません。実データで確かめると差は歴然です：

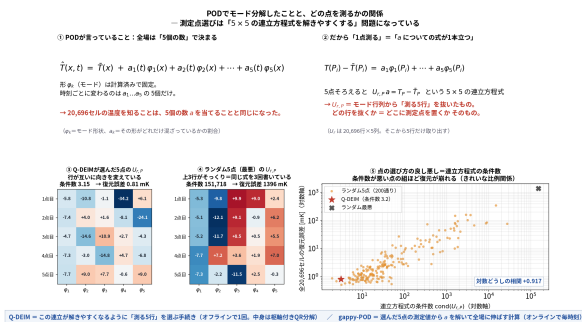


図 2: POD と測定点の関係

③ *Q-DEIM* が選んだ 5 点の $U_{r,P}$ は行が互いに向きを変えていて条件数 **3.15**、復元誤差 **0.81 mK**。④ ランダム 5 点の最悪例は上 3 行がそっくり (− 5.3, − 9.8, +9.9... と − 5.1, − 12.1, +9.1... と − 5.2, − 11.7, +8.5...) で条件数 **151,718**、復元誤差 **1,396 mK**。⑤ ランダム 200 通りを並べると、条件数と復元誤差は対数どうしの相関 **+0.917** できれいに並ぶ = 「点の選び方の良し悪し」は「連立の条件数」で測れている。

Q-DEIM は、この条件数になるべく良くなるように **5 行を貪欲に選ぶ**手続きです (中身は §4-2)。そして選んだあとの毎時刻の計算が **gappy-POD** ——これで 2 つの関係がつながりました。

■効果を温度 [K] で見る 選点の進行を測る「残差ノルム」は復元前のモード空間の無次元量なので、実際の温度で何 K になるのかを別に出しておきます:

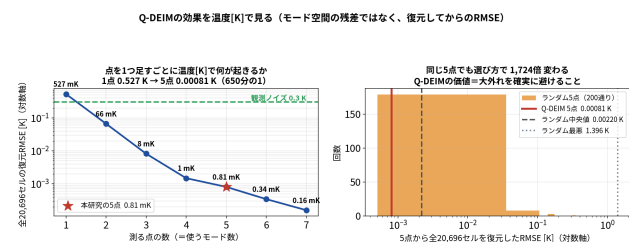


図 3: 点数ごとの温度 RMSE

左: 点を 1 つ足すごとの、全 20,696 セルの復元 *RMSE*。1 点 **0.527 K** → 2 点 **0.066 K** → 5 点 **0.00081 K** (650 分の 1)。観測ノイズ **0.3 K** を 2 点目で下回り、5 点では 2 桁下。右: 同じ「5 点」でもランダムに選ぶと中央値 **0.00220 K**、最悪 **1.396 K** (*Q-DEIM* の **1,724** 倍)。*Q-DEIM* の価値は「いつも最良」ではなく、大外れを確実に避けること。

まとめ (4-0) - **POD** が「全場は 5 個の数で決まる」という形を用意する - **Q-DEIM** が「その 5 個を解きやすくする 5 点」をオフラインで 1 回選ぶ - **gappy-POD** が「その 5 点の測定値から 5 個を解いて全場へ伸ばす」をオンラインで毎時刻やる
3 つは順番につながった一本の流れで、*Q-DEIM* と *gappy-POD* は設計と運用の関係にあります。

4-1. まず「なぜ点の選び方が問題になるか」

§3 で見たように、温度場は「平均場 + モードの足し合わせ」で表せます:

$$\hat{u} = \bar{u} + a_1\varphi_1 + a_2\varphi_2 + \dots + a_r\varphi_r = \bar{u} + U_r a.$$

ここで未知は モードの混ぜ具合 $a = (a_1, \dots, a_r)$ の r 個だけ。 r 個の未知を決めるには r 個の独立な測定があり、選点行列がフルランクなら係数を決められる ——これが「少数点で ROM を組める」理由です。 r 点 P で測ると、その点でのモードの値だけを並べた $r \times r$ 行列 $U_{r,P}$ を使って、連立方程式

$$U_{r,P} a = u_P - \bar{u}_P \Rightarrow a = (U_{r,P})^{-1} (u_P - \bar{u}_P)$$

を解くだけ (u_P = 選んだ点の測定値)。問題は、この連立が「ちゃんと解けるか」。

- 選んだ 2 点がモードから見て”そっくり” (例: どちらも「モード 1 が大きくモード 2 は小さい」) だと、2 本の式がほぼ同じ式になり、連立が解けない ($U_{r,P}$ が特異に近い)。測定に少しでもノイズが乗ると、復元が大暴れします。
- 選んだ点が互いに”別々のことを言う” (1 点はモード 1 を、別の点はモード 2 をよく映す) なら、連立はスッキリ解け、復元は安定。

図で見ると一目瞭然です。各セルを「そのセルでのモード 1・モード 2 の値」= モード指紋 (φ_1, φ_2) として平面に置きます:

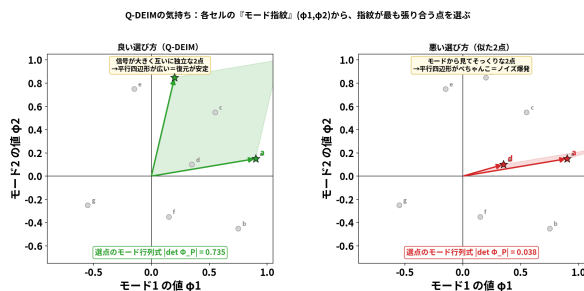


図 4: *Q-DEIM* の直感 (モード指紋と選点)

左 (良い): 指紋が直交気味の 2 点を選ぶと、原点からの 2 ベクトルが作る平行四辺形が広い = $|\det U_{r,P}|$ が大 (**0.735**) = 連立がよく解ける。右 (悪い): 指紋がそっくりな 2 点だと平行四辺形がぺちゃんこ = $\det$ が小 (**0.038**) = ノイズが爆発。つまり「選点でのモード行列式 (体積) が大きい点集合ほど、復元が安定」。

4-2. *Q-DEIM* = 「信号が大きく・互いに独立な点」を貪欲に選ぶ

では体積が最大の点集合をどう見つけるか。全組み合わせを試すのは大変なので、*Q-DEIM* は **貪欲法** (1 点ずつ最善を足す) で選びます。これが「モード行列の転置に列枢軸付き **QR** 分解をかける」の中身:

- 1 点目: 全セルの中で、モード指紋の長さ $\|\varphi_i\|$ が最大の点を選ぶ (一番強く場に効く点)。
- 2 点目: 1 点目の方向を全セルから差し引き (= 1 点目で説明できる分を消す)、残り (= 1 点目では分からない成分) が最大の点を選ぶ。
- これを r 点そろえるまで繰り返す。

「新しい点ほど、すでに選んだ点では分からない情報を最大にする」という素直な戦略です。上の図なら、まず一番長い指紋の **a**（モード 1 が最大）を選び、次に **a** の向きを消して残りが最大の **h**（モード 2 が最大）を選ぶ → 直交気味の良いペアになります。列軸 QR はこの手順を数値的に安定に行うだけで、魔法ではありません。

4-3. 手計算：1 点で場が復元できることを確かめる（文字式）

いちばん簡単なモード **1** 本（ $r = 1$ ）の場合で、上の手順を最後までやってみます。 $U_1 = \varphi_1 = a/\|a\|$ (§3 のランク 1 例)。軸軸 QR は絶対値が最大の成分を最初の点に選ぶので、 $a_A > a_B > a_C$ より選ばれるのはセル **A**（一番よく振れる点）。「振れの大きい場所を測れ」という直感どおりです。

では本当に **A** の 1 点だけで場全体が戻るか。ある時刻 t_k で **A** だけ測ると、測定値の変動は $u_A(t_k) - \bar{u}_A = a_A g_k$ 。選点行は $U_{1,A} = \varphi_1[A] = a_A/\|a\|$ なので、係数は

$$\alpha = (U_{1,A})^+ (u_A - \bar{u}_A) = \frac{\|a\|}{a_A} \times a_A g_k = \|a\| g_k.$$

これを全場に伸ばすと、

$$\hat{u} = \bar{u} + \varphi_1 \alpha = \bar{u} + \frac{a}{\|a\|} \times \|a\| g_k = \bar{u} + a g_k.$$

一方、真値は §3 の定義そのまま $u(t_k) = \bar{u} + a g_k$ 。つまり

$$\hat{u} = \bar{u} + a g_k = u(t_k)$$

測っていない **B・C** まで、真値にピッタリ一致。 $\|a\|$ がきれいに約分して消えるのがミソで、ランク **1** なら 1 点測るだけで場が完全復元できる、というのが文字式で最後まで確認できました。本物は 2~3 モードなので、同じ理屈で数点あれば場が復元できます。これが「少数の代表点で ROM を組める」根拠です。

■実データで貪欲選択を 1 ステップずつ追う §4-2 の手順 (①いちばん強い点を選ぶ ②その方向を差し引く ③繰り返す) が、実際にそのとおり動いているかを 20,696 セルで確認します (run/qdeim_explained.py) :

```
W = Ur.copy() # Ur = 先頭 5 モード
↳ (20696×5)
for k in range(5):
    norms = np.linalg.norm(W, axis=1) # 各セルに「残って
    ↳ いる情報」の大きさ
    p = int(np.argmax(norms)) # ← 最大の点を選ぶ
    v = W[p] / np.linalg.norm(W[p])
    W -= np.outer(W @ v, v) # ← その方向を全セルか
    ↳ ら差し引く
```

実行すると、選ばれる点と残差の減り方はこうなりました:

順番	選ばれたセル	座標 [mm]	そのときの残差ノルム	差し引いた後の最大残差
1 点目	7634	(21.3, - 1.6, 51.6)	0.03688	0.02657
2 点目	7924	(- 14.1, 33.1, 51.6)	0.02657	0.01922
3 点目	7765	(36.5, 6.9, 50.8)	0.01922	0.01729
4 点目	859	(35.1, 0.8, 0.8)	0.01729	0.01425
5 点目	733	(- 35.1, 0.8, 0.8)	0.01425	0.00000

読み取れることが 3 つあります:

- 残差が単調に減る : 0.0369 → 0.0266 → 0.0192 → 0.0173 → 0.0143。1 点選ぶごとに「まだ分かっている情報」が着実に減っています。
- **5 点目で残差がちょうど 0** : 5 モードぶんの情報を 5 点で完全に張り切ったということ。これが「 r モードには r 点」の意味です。
- 選ばれる順番が意外 : 1 点目はヒータ最近傍ではなく中央高さの点 (21.3, - 1.6, 51.6)。ヒータ側の点 (36.5, 6.9, 50.8) は 3 番目です。Q-DEIM は「一番熱い点」ではなく「場全体をいちばんよく代表する点」を選ぶため、この直感とのズレは押さえておくべきところです。

scipy の軸軸 QR と一致するかも確認しました:

手で追った貪欲選択: [7634, 7924, 7765, 859, 733]

scipy 軸軸 QR : [7634, 7924, 7765, 859, 733] → 完全一致

つまり qr(..., pivoting=True) は、上のループを数値的に安定にやっているだけです。

■適当に 5 点選んだ場合と比べる 「わざわざ Q-DEIM を使う価値があるのか」を、ランダムな 5 点 200 通りと比較しました。5 点から全 **20,696** セルを復元したときの RMSE です:

選び方	復元 RMSE
Q-DEIM	0.00081 K
ランダム (中央値)	0.00220 K
ランダム (最良)	0.00051 K
ランダム (最悪)	1.396 K

ここが実務的に重要です:

- 中央値との比較では **3 倍** の差にすぎず、運が良ければランダムでも **Q-DEIM** を上回る (最良 0.00051 K)
- しかし最悪のランダム配置は **1.396 K = Q-DEIM の約 1700 倍** の誤差
- 選点行列の行列式でも Q-DEIM 4.6×10^{-9} に対しランダム中央値 3.1×10^{-11} (約 150 倍)

この 2 つを図にするとこうなります:

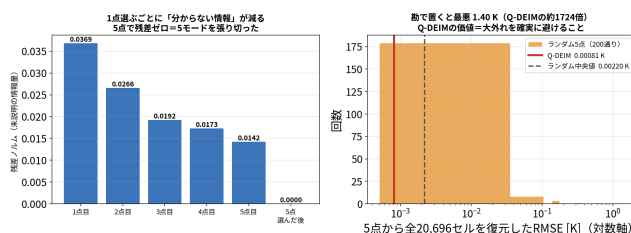


図 5: Q-DEIM の貪欲選択と、ランダム配置との比較

左：1 点選ぶごとに残差が減り、5 点で完全にゼロになる。
 右：ランダム 5 点 200 通りの復元 RMSE 分布（対数軸）。多くは Q-DEIM の近くに集まるが、裾が 1.4 K まで伸びているのが危険。

→ Q-DEIM の価値は「平均的に少し良い」ことではなく、「大外れを確実に避ける」ことです。勘で置くと、たまたま似た情報しか持たない点ばかりを選んでしまい、復元が桁で崩れる危険があります。

4-4. 実際に選ばれた 5 点

本ケース (20,696 セル、モード 5 本) に Q-DEIM をかけると、上の貪欲手順で次の 5 点選ばれます：

POD+Q-DEIM で選んだROM代表点（勘でなくデータから選定）
 温度場は2モードで100%説明→少数点で表せる

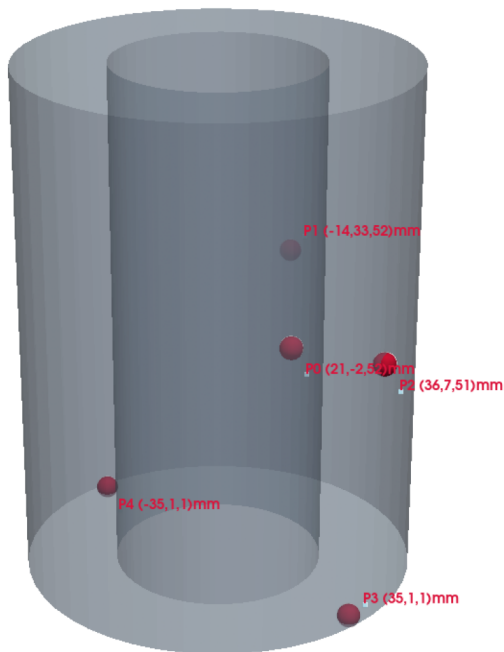


図 6: POD + Q-DEIM で選んだ ROM 代表点 (3D)

5 点を個別に読み取れるよう、温度感度 ($\partial T / \partial Q$) と実際の変位観測点を同じ座標系に重ねた図も示します。

■「高感度／低感度」はどういうロジックで決めているか
 本研究では温度センサと変位センサで、感度の定義と決め方が違います。混同しやすいので、レシピ形式でまとめます。

① 温度センサの高/低感度 = 発熱への温度感度 $\partial T / \partial Q$

「ヒータの発熱が変わったとき、温度がよく反応する場所はどこか」

1. 発熱を 10% 増やして ($q_{\text{scale}}: 1.0 \rightarrow 1.1$) ROM をもう一度回す
2. 加熱ピーク ($t = 300 \text{ s}$) で、各点の温度上昇の差をとる：

$$s_i = \frac{T_i(1.1Q) - T_i(Q)}{0.1Q}$$
3. 大きい順に並べる → 最大 = 高感度 (P2 = ヒータ最近傍、5.7)、最小 = 低感度 (P4 = 反対側の底面、2.6)

物理的な理屈：熱源に近いほど発熱変化が薄まらずに届くので反応が大きい。遠い点は拡散でならされ、反応が鈍い。

② 変位センサの高/低感度 (高 W/低 W) = 温度への変位感度

$$W = K_s^{-1} H_T$$

「温度が変わったとき、変位がよく動く場所はどこか」

1. FrontISTR の熱弾性 ($K_s u = H_T T$) を 1 度解いて、温度 → 変位の行列 W を作る
2. 候補点ごとに W の行ノルム (= その点の変位が温度場にどれだけ反応するか) を計算
3. 大きい順に並べる → 高 W = 上端の点、低 W = 底面の点

物理的な理屈：底面は固定なので、そこから遠い上端ほど各層の伸びが累積して現れる (てこの先端ほど動く)。底面近くは拘束されていてほとんど動けない = 測っても情報が少ない。

まとめ表：

温度センサ		変位センサ	
感度の定義	$\partial T / \partial Q$ (発熱 → 温度)	$W = K_s^{-1} H_T$ の行ノルム (温度 → 変位)	
決め方	Q を 10% 振って温度差を見る	FrontISTR を 1 回解いて W の行を比べる	
高感度の場所	ヒータ最近傍 (P2)	上端 (固定面から最遠)	
低感度の場所	反対側の底面 (P4)	底面 (固定で動けない)	
どちらも意味は「測る量が、知りたい変化によく反応する場所」を選ぶ、という同じ発想			

(詳しい導出は blog_002 §4 と 105_0_sensor_placement_sensitivity/docs/03_sensor_design_guide.md にあります。)

観測点の位置: 温度センサ(ROM代表点を dT/dQ で色付け)と変位観測点(上面A/O)
 温度センサは高感度ノード P2 ($dT/dQ=5.7$)、避けるのは低感度 P4 ($dT/dQ=2.6$)

Disp O (-X top)

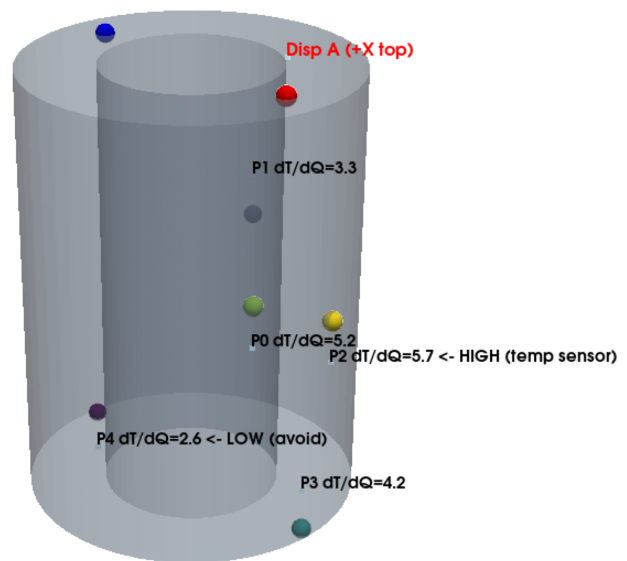


図 7: 5 温度点と変位観測点 A/O の位置

点	座標 (x, y, z) [m]	役割
P0	(0.0213, -0.0016, 0.0516)	中央高さ・温度代表点
P1	(-0.0141, 0.0331, 0.0516)	中央高さ・反対側の温度代表点
P2	(0.0365, 0.0069, 0.0508)	$\partial T / \partial Q$ 最大の温度観測候補
P3	(0.0351, 0.0008, 0.0008)	底面・ヒータ側の温度代表点
P4	(-0.0351, 0.0008, 0.0008)	低感度点 (比較用)

赤い Disp A と青い Disp O は温度点ではなく、FrontISTR で取得する上面の変位観測位置です。つまり P0~P4 は温度場を表

す点、A/O は変位を測る点で、役割が異なります。

勘の流用点（すべて中央高さ）と違い、底面 (P3,P4) も選んで場を広くカバーしている。「I 点目＝一番効く点 →以降＝それまでで分からない情報が最大の点」を繰り返した結果で、上下・ヒータ側/反対側にバラけて場全体を押さえているのが分かります。「勘」ではなく「データから系統的に」選んだ点であることがポイント。

5. 実データの POD 結果（図で確認）

5-0. 小例から実データへーモードは「空間の分布（型）」

§3 の小例は $X = ag^T$ とランク 1 で、モードは 1 本 ($\varphi_1 = a/\|a\|$) = 「A 強・B 中・C 弱」という 3 セル上の分布でした。実物の温度場は片側加熱で少し複雑なので、1 本では足りず数本のモードが要ります。ただしやることは全く同じ: 20,696 セル × 121 時刻のスナップショット行列を SVD すると、モード $\varphi_1, \varphi_2, \varphi_3, \dots$ が出来ます。

ここで大事なのは、各モード φ_k は「1 枚の空間分布（円筒のどこがどれだけ動くかの地図）」だということ。小例の $a = (a_A, a_B, a_C)$ を、3 セルから 20,696 セルへ拡張したものです。そして温度場は「平均場 + モード分布の足し合わせ」で表せます:

$$\hat{u}(t) = \bar{u} + a_1(t) \varphi_1 + a_2(t) \varphi_2 + \dots$$

(φ_k = 時間によらない空間の型、 $a_k(t)$ = その型が時刻ごとにどれだけ混ざるか)。

5-1. 各モードの分布を読む

下の図が、実際に出了 平均場と上位 3 モードの空間分布 です:

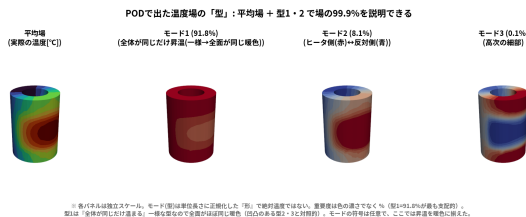


図 8: POD で出た温度場のモード

各パネルの色は「そのモードが円筒のどこで正/負か」を表します。読み方:

- 平均場: 定常的な下地の温度分布 (ヒータ側が高い、実際の℃)。ここからの変動をモードが表す。
- モード 1 (91.8 %): ほぼ一様な型 = 「全体が同じだけ昇温する」分布 (全面がほぼ同色)。小例の「全セルが一斉に上下する」に対応。最も支配的。
- モード 2 (8.1 %): ヒータ側が+・反対側が- の偏り分布 = 片側加熱の非対称を表す型。
- モード 3 (0.1 %): さらに細かい残り (上下・角度方向の微調整)。ほとんど効かない。

つまり「モードを分解した」結果は、“一様昇温” + “ヒータ

側/反対側の偏り” + “微細な残り” という 分布の重ね合わせに温度場を分けた、ということです。

コラム: モードの意味は”後付け” / 事前にわかる?

モードはあらかじめ意味を決めて作るものではなく、データ (SVD) から自動で出てくるものです。並び順は必ずエネルギー (変動の大きさ) の大きい順: モード 1 = 一番大きい変動の型、モード 2 = それを除いた残りで最大 (モード 1 と直交)、... という具合。「一様昇温」「片側の偏り」という物理的な意味は、出てきた分布を見て人間が後から解釈したものです。SVD はヒータの存在も物理も知らず、ただ「分散が最大の直交方向」を順に拾っただけ。物理がたまたま解釈しやすい形にしてくれている、という順序です。事前に確定はできません (= データ駆動)。CFD を回してスナップショットを SVD して初めて、分布の形も エネルギー比 (91.8 / 8.1 / 0.1 %) も決まります。ただし対称性・物理から”顔ぶれ”の見当はつきます: 片側加熱の対称な物体なら、経験的に「①全体昇温 → ②ヒータ側と反対側の dipole (偏り) → ③高次の細かい模様」となりがちで、今回もそのとおりでした。なおモードの符号は任意 (φ と $-\varphi$ は同じ意味) で、エネルギーが近い 2 モードは順番が入れ替わることもあります。加熱位置や境界条件が変わればモードも変わる (その問題固有)。

5-2. エネルギー ーどのモードが何割を説明するか

各モードの エネルギー比は「場の変動のうち、その型が何割を説明するか」(小例では 1 モードで 100 % でした):

モード	エネルギー比	累積
モード 1	91.81 %	91.81 %
モード 2	8.08 %	99.89 %
モード 3	0.11 %	99.996 %

2 モードで 99.9 % = 温度場は実質 2 自由度 (一様昇温 + 片側の偏り) でほぼ説明しきれる。だから数点の代表点で ROM を組めば十分 = §4 の Q-DEIM で少数点を選ぶ根拠になります。

5-3. モードを足すほど真値に近づく

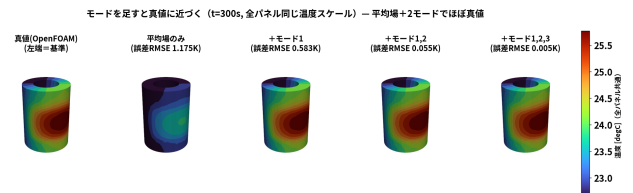


図 9: モードを足すと真値に近づく

平均場のみ (誤差 1.175K) → +モード 1 (0.583K) → +モード 1,2 (0.055K) → +モード 1,2,3 (0.005K)。モードを 1 つ足すごとに桁で誤差が縮む = POD が効率よく場を捉えている証拠。

5-4. 5 点から温度分布を復元する ー遠いセルも「モード經由」で決まる

データ同化が 5 点の温度を出したら、そこから全 20,696 セルの温度分布を復元できます (gappy-POD)。手順は 2 段:

1. 5 点の測定値からモードの混ぜ具合を解く: $a = U_{5,P}^{-1} (T_5 - \bar{u}_5)$ ($U_{5,P}$ = 5 点でのモード値の 5×5 行列)
2. 全セルを復元: $\hat{T}_i = \bar{u}_i + \sum_k a_k \varphi_{k,i}$ (各セルは自分のモード値 $\varphi_{k,i}$ を使う)

■選択行列から復元式まで省略せずに書く T_r は同化後の 5 代表点温度 (全 5 点の直接測定値ではありません)。全場を $T_{\text{full}} \approx \bar{u} + \Phi a$ と表すと、

$$T_r = P T_{\text{full}} \approx P \bar{u} + P \Phi a.$$

平均を引いて $(P\Phi)a = T_r - P\bar{u}$ を解き、全場の式へ戻すと、

$$a = (P\Phi)^+ (T_r - P\bar{u}), \quad \hat{T}_{\text{full}} = \bar{u} + \Phi (P\Phi)^+ (T_r - P\bar{u}).$$

$P\Phi$ は 5×5 。フルランクなら逆行列と疑似逆行列は一致します。一般には、選んだ点での残差二乗和を最小にする係数を疑似逆行列で求めます。観測点数不足やモード外の温度変化があれば、全場を一意・正確に復元できるとは限りません。

「gappy」は全場の一部にしか値がない状態を指します。この処理は距離補間ではなく、学習した空間モードによる復元です。

■この式の読み方ー「モードの割合」で各点の温度が決まる 各時刻のモード係数 a_k が、固定したモードをどれだけ足すかを決めます。係数は負にもなり、合計 1 の割合ではありません。式の 2 つの部品は役割がはっきり違います:

- $a_k(t)$... モードの割合 (混ぜ具合)。時刻ごとに 1 組の数 ($a_1, a_2, \dots$) で、全セルで共通。5 点の測定から決まる。
- $\varphi_{k,i}$... そのセルがモード k をどれだけ受け取るか (=セルの指紋)。POD で 1 度だけ決まり、時間によらず固定。

だから各セルの温度は「下地 $\bar{u}_i$ + (全体の割合 a_k) × (そのセルの受け取り $\varphi_{k,i}$) の合計」。例えるならミキシングボード: モードが 1 本ずつのスライダー (空間パターン) で、 a_k がその上げ幅。5 点で「今どのスライダーがどれだけ上がっているか」を読み取れば、あとは各セルが自分の感度でその音を受け取るだけです。

1 セルで確認 (上の C で、割合が $a = (1, 0.5)$ のとき):

$$\hat{T}_C = \bar{u}_C + a_1 \varphi_{1,C} + a_2 \varphi_{2,C} = 20.75 + 1 \times 0.3 + 0.5 \times 0.7 = 21.4.$$

もし別の時刻で割合が $a = (2, 0.3)$ に変われば、同じ C でも

$$\hat{T}_C = 20.75 + 2 \times 0.3 + 0.3 \times 0.7 = 21.56$$

と変わります。つまり 温度の時間変化は「割合 $a(t)$ の変化」

で表される (下地とモードの形は固定、動くのは割合だけ)。そして全セルが同じ a を共有するから、5 点で a さえ決めれば、残り全部が決まる ーこれが「5 点から分布を作る」の芯です。

ここが肝心: 遠いセルは「近くの 5 点からの距離で内挿」しているわけではありません。5 点で決めた係数 a を、そのセル自身のモード指紋 $\varphi_{k,i}$ に掛けて計算します。だから「5 点から遠い反対側のセル」も、モードの形を通じてちゃんと値が決まります。

具体例 (2 モードで確認): 3 セル A,B,C、平均 $\bar{u} = (23, 21.5, 20.75)$ 、モード

$$\varphi_1 = (0.8, 0.5, 0.3), \quad \varphi_2 = (0.1, -0.6, 0.7).$$

真の混ぜ具合を $a = (1, 0.5)$ とすると、真の温度は $u = \bar{u} + a_1 \varphi_1 + a_2 \varphi_2 =$

$$u_A = 23.85, \quad u_B = 21.7, \quad u_C = 21.4.$$

いま A・B だけ測り、C は遠くて測らないとします。選点行列と測定値は

$$U_{2,P} = \begin{pmatrix} 0.8 & 0.1 \\ 0.5 & -0.6 \end{pmatrix}, \quad u_P - \bar{u}_P = \begin{pmatrix} 23.85 - 23 \\ 21.7 - 21.5 \end{pmatrix} = \begin{pmatrix} 0.85 \\ 0.2 \end{pmatrix}.$$

逆行列 ($\det = 0.8 \cdot (-0.6) - 0.1 \cdot 0.5 = -0.53$) を掛けて混ぜ具合を復元:

$$a = U_{2,P}^{-1} \begin{pmatrix} 0.85 \\ 0.2 \end{pmatrix} = \frac{1}{-0.53} \begin{pmatrix} -0.6 & -0.1 \\ -0.5 & 0.8 \end{pmatrix} \begin{pmatrix} 0.85 \\ 0.2 \end{pmatrix} = \frac{1}{-0.53} \begin{pmatrix} -0.53 \\ -0.265 \end{pmatrix} = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}.$$

この $a = (1, 0.5)$ を、一度も測っていない C の指紋 $\varphi_{\cdot,C} = (0.3, 0.7)$ に掛けると:

$$\hat{T}_C = \bar{u}_C + a_1 \varphi_{1,C} + a_2 \varphi_{2,C} = 20.75 + 1 \times 0.3 + 0.5 \times 0.7 = 21.4 = u_C.$$

遠い C が、測らずにピタリ復元できました。距離で内挿したのではなく、「5 点で決めた $a \times C$ のモード値」で決まっている ーこれが「5 点から分布を作る」の正体です。

動く絵で見ると (各時刻で ROM の 5 点温度から全場を復元):

t = 304 s (cooling (heater OFF))
5 sensors -> full field (POD)

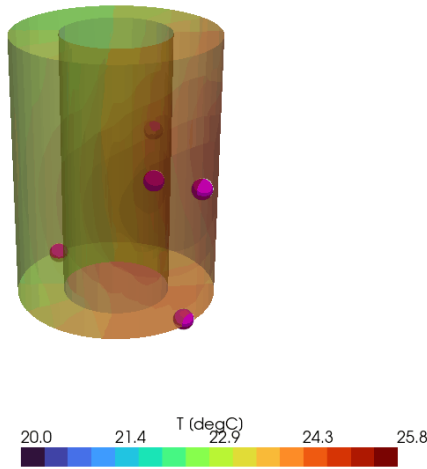


図 10: 5 点→全温度場の復元アニメ

ピンク色の球（マゼンタ）＝復元に使う 5 点（温度コンタは半透明で内部の点も見える）。各時刻で ROM が出す 5 点温度からモード係数 $a(t)$ を解き、全 20,696 セルを復元。ヒータ側が熱く反対側が冷たい分布が、加熱→（300 秒で OFF）→冷却と動く。5 点しか使っていないのに、面全体の温度分布が出るのが POD の効きどころです。

6. 選んだ 5 点で ROM を組み、校正する

6-1. 一般化した集中定数モデル

ノードは §4 で選んだ 5 点そのもの (P0~P4)。その各ノード i について、次の 1 本を立てます ($i = 0, 1, 2, 3, 4$ の 5 本の連立式)：

$$C_i \frac{dT_i}{dt} = \sum_{j \neq i} K_{ij}(T_j - T_i) + q_i(t) - h(T_i - T_{\text{air}}).$$

右辺は左から「他ノードからの熱流入」「発熱」「空気への放熱」の 3 項です。

- C_i ... ノード i の熱容量 [J/K]
- K_{ij} ... ノード i, j 間コンダクタンス [W/K] (対称・全点对、5 点なら 10 本)
- q_i ... 発熱。ヒータ最近傍ノード P2 だけに投入 (他は $q_i = 0$)
- h ... 放熱係数 [W/K]

「どのノードの式か」を 3D で示すと (ParaView 風)：

ROM のノード=POD+Q-DEIM 選定 5 点。各ノードで集中定数の式を解く
辺の太さ=コンダクタンス K_{ij} (P0-P2 が最強)。オレンジ=ヒータ発熱ノード P2 に q を投入

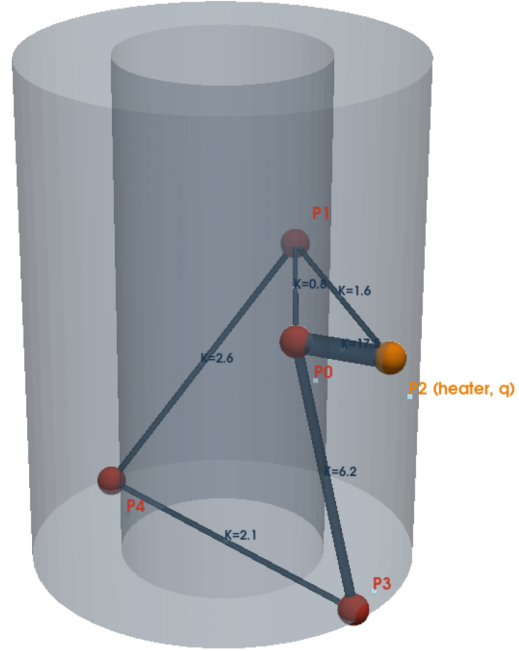


図 11: ROM の 5 ノードと結合ネットワーク

ノード = $POD + Q-DEIM$ 選定 5 点。各辺の太さが校正で決まったコンダクタンス K_{ij} で、**P0-P2** が最強 ($K = 17.5 \text{ W/K}$) = ヒータ側どうしが強く熱をやり取りする。オレンジの P2 がヒータ発熱ノード (q を投入)、そこから $P0 \rightarrow P3 \rightarrow \dots$ と全ノードへ熱が伝わる。底面 P3・P4 は弱い結合で、冷えやすい末端。

各ノードの熱容量 C_i ・結合 K_{ij} ・放熱 h は次の 6-2 で実データに合わせて決めます。行列形にすると線形システムになり、RK4 で一括前進積分できます (これが約 31ms の正体)。

1D-CAE との関係：この「熱容量ノードをコンダクタンスで結び、発熱と放熱を付けた熱 RC 回路」は、まさに 1D-CAE (Modelica / Simulink や熱回路網法) でやる集中定数モデルと同じ枠組みです。違いは 2 点だけ：教科書的な 1D-CAE がノードを勘で置き、係数を幾何の公式 (例 $K = kA/L$, $C = \rho cV$) で決めるのに対し、ここではノードを **POD + Q-DEIM** でデータから選び、係数を **3D 連成解析** にフィットします。つまり“点も係数もデータ駆動の **1D-CAE**”です。

6-2. 係数 K_{ij} はどう決める？－幾何でなく校正 (最小二乗)

K_{ij} は公式 $K = kA/L$ では計算しません。「5 点の ROM 温度履歴が、OpenFOAM の 5 点温度履歴に一致するように」未知パラメータをまとめて逆算 (同定) します。未知は

$$\theta = [\underbrace{C_1, \dots, C_5}_5, \underbrace{K_{12}, K_{13}, \dots, K_{45}}_{10}, \underbrace{h}_1],$$

の合計 16 個。

ROM を θ で前進積分して 5 点温度 $T_p^{\text{ROM}}(t; \theta)$ を作り、OpenFOAM の $T_p^{\text{OF}}(t)$ との差を最小化します ($C, K, h \geq 0$ の範囲制約つき、`scipy.optimize.least_squares`)：

$$\hat{\theta} = \arg \min_{\theta} \sum_{p=1}^5 \sum_t (T_p^{\text{ROM}}(t; \theta) - T_p^{\text{OF}}(t))^2.$$

実際のコードはこれだけです（run/build_calibrate_rom.py の要点）：

```
from scipy.optimize import least_squares
import numpy as np

# Yobs : OpenFOAM の 5 点温度履歴 (nt,5) ← 校正の目標
# ts : 時刻列, DT: 内部刻み, heat_node: 発熱を入れるノード
#   (=P2)

def forward(theta):
    """ θ=[C(5), K 上三角 (10), h(1)] で ROM を前進積分し、5 点
    ↳ 温度履歴を返す。 """
    C = theta[:5]
    K = tri_to_matrix(theta[5:15], 5) # 10 個 → 対称 5x5 コン
    ↳ ダクタンズ行列
    h = theta[15]
    T = np.full(5, T_air); Y = [T.copy()]
    for a, b in zip(ts[:-1], ts[1:]): # 各区間を RK4 で前進
        T = integrate_rom(T, C, K, h, q=1.0, heat_node=heat_node,
        ↳ t0=a, t1=b, dt=DT)
        Y.append(T.copy())
    return np.array(Y) # (nt,5)

def resid(theta):
    return (forward(theta) - Yobs).ravel() # OpenFOAM との差
    ↳ (残差ベクトル)

x0 = np.r_[np.full(5, 120.0), np.full(10, 1.5), 0.3] # 初期推定
↳ [C,K,h]
lb = np.r_[np.full(5, 1.0), np.full(10, 1e-4), 1e-4] # 下限 (すべて 0)
ub = np.r_[np.full(5, 5000.0), np.full(10, 200.0), 50.0] # 上限

sol = least_squares(resid, x0, bounds=(lb, ub)) # ← 16 個
↳ をまとめて同定
C, K, h = sol.x[:5], tri_to_matrix(sol.x[5:15], 5), sol.x[15]
```

やっていることは「 θ を少しずつ変えて forward の出力を OpenFOAM に近づける」だけ。least_squares が残差の二乗和を最小にする θ を自動で探します（16 個の未知を一括同定）。

Yobs の正体 = **OpenFOAM** の結果です。実際には

```
ts, C_all, X = load_snapshots() # OpenFOAM の各時刻
↳ solid/T を読む (20696×121)
Yobs = X[cells, :, T] # そのうち Q-DEIM 選定 5 点の
↳ 温度履歴だけ抜き出す
```

のように、load_snapshots() が **CHT** 結果を読み込み、5 点ぶんを取り出したものが校正の目標。OpenFOAM の結果は ① **POD + Q-DEIM**（点選び）と ② この校正の両方の材料になっています——これが「オフラインで CFD が必須」（§1-1）の正体です。

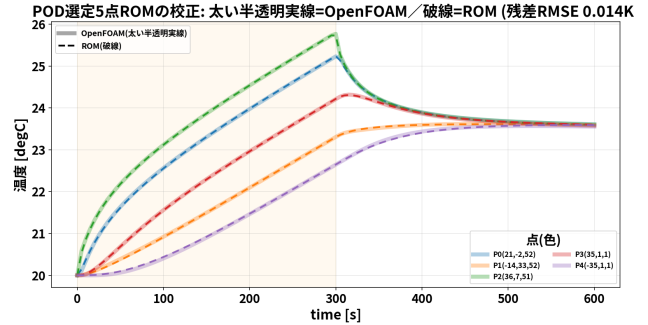


図 12: POD 選定 5 点 ROM の校正結果

太い半透明線 = *OpenFOAM*、破線 = *POD* 選定 5 点 *ROM*。ほぼ完全に重なる（残差 *RMSE* 0.014 K）。

同定された K_{ij} (W/K) は、幾何でなく **3D** の伝熱がそう振る舞うように決まります：

結合	K_{ij}	意味
P0-P2	17.5	ヒータ側どうし = 最強
P0-P3	6.2	ヒータ側の縦方向
P1-P4, P3-P4	2.6, 2.1	反対側・底面
P1-P2	1.6	中央どうし
その他	≈ 0	弱く、実質つながっていない

P0-P2 が突出 = ヒータ近傍の強い熱交換を反映。“つながりの強さ”がデータから浮かび上がるのが、勘で組む 1D-CAE との違いです。

- 当てはめ残差 **RMSE = 0.014 K**（OpenFOAM をほぼ完全に再現）
- 全熱容量 $\sum C_i = 1211 \text{ J/K}$ は、鋼 2.49 kg の $\rho c_p V \approx 1195 \text{ J/K}$ とほぼ一致（数字合わせでなく物理的にも妥当。フィットなのに物理量として筋が通る）

7. 温度が求まったら — 実形状にマッピングして FrontISTR で変位を出す

ROM + データ同化で温度が求まっても、本当に知りたいのは変位（熱膨張・反り）です。温度 → 変位は blog_001 と同じ FrontISTR ですが、ここでは「温度をどう実形状へ渡すか」を数式で押さえます。

7-0. 変位を「観測」として使う手続き 一状態に入れずに観測演算子で作る

先に、いちばん分かりにくいところを片付けます。「変位計の値をデータ同化にどう入れるのか」です。

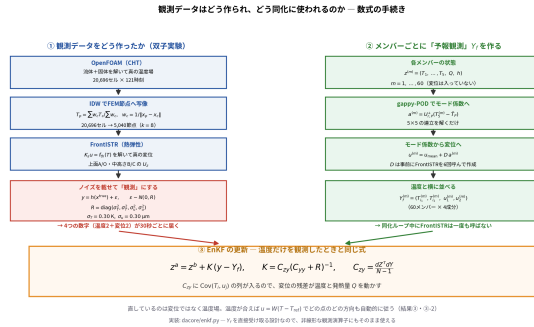


図 13: 観測の作り方と使い方

①観測データの作り方 (双子実験) / ②予報観測 Y_f の作り方 / ③ $EnKF$ の更新式。

■まず結論

変位は状態ベクトルに入れません。状態はあくまで $z = (T_1, \dots, T_5, Q, h)^T$ の 7 次元のままです。変位は観測演算子の中で、そのつど温度から計算します。

これができる理由は、温度から変位への写像が線形だからです (§7-2)。

■手続き (4 ステップ) $EnKF$ の各サイクルで、60 メンバーそれぞれについて次を実行します。

ステップ 1: そのメンバーの 5 点温度を取り出す

$$T_5^{(m)} = (T_1^{(m)}, \dots, T_5^{(m)}) \quad m = 1, \dots, 60$$

ステップ 2: gappy-POD でモード係数にする (§4-0 と同じ計算)

$$a^{(m)} = U_{r,p}^+(T_5^{(m)} - \bar{T}_p) \quad (5 \times 1)$$

ステップ 3: モード係数から変位を出す

$$u^{(m)} = u_{\text{mean}} + D a^{(m)} \quad (2 \times 1)$$

ここで D は $FrontISTR$ で事前に作った定数行列です (2×5)。



図 14: D 行列とは何か

①線形性で $u = W\bar{T} + \sum a_k(W\phi_k)$ と分けられ、 $W\phi_k$ は a_k に依存しない → 先に計算して保存できる。② $FrontISTR$ を 6 回 (平均場 + 5 モード) 呼ぶだけ。③実際の D (ϕ_2 の列は A が、 O が + = 逆向きで反りを生む)。④ 60 メンバー分の変位計算が 5.2 μs 。毎サイクル FEM を呼べば数日かかる。

☒ この手が使えるのは熱弾性が線形だからです。塑性・クリープ・接触があると W が状態に依存するので、重ね合わせが成り立ちません。

各列 $D[:,k]$ が「モード ϕ_k が 1 単位混ざったときの上面 A/O の変位」で、同化を始める前に $FrontISTR$ を 1 + 5 = 6 回だけ呼んで作ってあります (§7-2)。

重要: 同化のループ中に $FrontISTR$ は一度も呼びません。呼ぶのは事前の 6 回だけ。あとは 2×5 の行列を掛けるだけなので一瞬です。

ステップ 4: 観測予報 Y_f に並べる

温度 2 点と変位 2 点を観測するなら、メンバーごとに 4 つの値を並べます:

$$Y_f^{(m)} = \underbrace{(T_{i_1}^{(m)}, T_{i_2}^{(m)})}_{\text{温度 2 点}}, \underbrace{(u_{i_1}^{(m)}, u_{i_2}^{(m)})}_{\text{変位 2 点}}$$

実装はこれだけです:

```
def disp_of_T5(T5):
    # (...,5) -> (...,2)
    a = (T5 - mean[pod_cells]) @ UP_pinv.T # ステップ 2:
    # gappy-POD
    return uz_mean + a @ Dmode.T # ステップ 3: モード
    # 変位

# ステップ 4: 温度と変位を横に並べて観測予報にする
Yf = np.column_stack([Z[:, nodes], # 温度 2 点
                      disp_of_T5(Z[:, :NPT])]) # 変位 2 点
Z = enfk_update(Z, y, None, Rd, rng, inflation=INFL, Yf=Yf)
```

■なぜこれで温度が直るのか $EnKF$ の更新式は温度だけを観測したときとまったく同じです:

$$z^a = z^b + K(y - Y_f), \quad K = C_{zy}(C_{yy} + R)^{-1}$$

変わるのは $C_{zy} = \text{Cov}(z, Y_f)$ の中身だけ。 Y_f に変位が入っているので、温度と変位の共分散 $\text{Cov}(T_i, u_j)$ が アンサンブルから自動的に計算されます。

$$C_{zy} = \begin{pmatrix} \text{Cov}(T_1, T_{i_1}) & \cdots & \text{Cov}(T_1, u_1) & \text{Cov}(T_1, u_2) \\ \vdots & & \vdots & \vdots \\ \text{Cov}(Q, T_{i_1}) & \cdots & \text{Cov}(Q, u_1) & \text{Cov}(Q, u_2) \end{pmatrix}$$

赤い列があるから、変位の残差 $y_u - u^b$ が温度と発熱量を引っばります。仕組みは blog_003 §4-0 の「温度しか測らなくても Q が動く」とまったく同じで、測っていない量でも、測った量と相関していれば更新されるという一点に尽きます。

■非線形な観測でも同じ形で書ける `enkf_update` は観測行列 H ではなく 予報観測 Y_f をそのまま受け取る設計にしています (`dacore/enkf.py`)。

```
def enkf_update(Zf, y, H, R, rng, inflation=1.0, Yf=None):
```

```
...
if Yf is None:
    Yf = Zf @ H.T    # 線形観測ならここで作る
```

今回の変位写像はたまたま線形 ($u = u_{\text{mean}} + Da$) ですが、メンバーごとに実ソルバを通すような非線形観測でも、 Y_f を渡せば同じ式で動きます。実際 104 では、各メンバーの温度場を `FrontISTR` に通した変位を Y_f にしていました (そのぶん 1 サイクルが分オーダーになり、ROM 化の動機になりました)。

7-1. 温度を実形状 (FEM メッシュ) へマッピング 一逆距離加重

ここからは空間の移し替えです。5 点→全セルの復元は §5 の `gappy-POD` で済んでいます。その後、CFD セルと FEM 節点の位置の違いを補間します。以下は `IDW` の説明で、すべての図が同じ補間実装という意味ではありません。例えば未観測 C/D の検証スクリプトは最近傍セルを使います。各図の再現スクリプトで確認してください。

CFD (または `POD` 復元) の温度はセル中心 x_c の値。いっぽう `FrontISTR` が要るのは節点 x_p の温度です。そこで各節点に、近い k 個のセルから 逆距離加重 (`IDW`) で内挿します:

$$T_p = \frac{\sum_{c \in \mathcal{N}_k(p)} w_{pc} T_c}{\sum_{c \in \mathcal{N}_k(p)} w_{pc}}, \quad w_{pc} = \frac{1}{\|x_p - x_c\|}.$$

近いセルほど重み w_{pc} が大きい、という素直な加重平均です (節点がセル中心と一致したらそのセル値をそのまま採用)。 $\mathcal{N}_k(p)$ は節点 p に近い k 個のセル。これで温度分布が実物形状の節点に乗ります。

具例 ($k=3$): ある節点 p の近くに 3 つのセルがあり、距離が $d_1=1, d_2=2, d_3=4$ mm、温度が $T_1=25, T_2=23, T_3=21$ °C だったとします。まず重みは距離の逆数 (近いほど大きい / 半分の距離なら 2 倍の重み):

$$w_1 = \frac{1}{d_1} = 1, \quad w_2 = \frac{1}{d_2} = 0.5, \quad w_3 = \frac{1}{d_3} = 0.25.$$

これを式に入れて、分子 (重み付き和) と分母 (重みの和) を計算すると:

$$T_p = \frac{w_1 T_1 + w_2 T_2 + w_3 T_3}{w_1 + w_2 + w_3} = \frac{1 \times 25 + 0.5 \times 23 + 0.25 \times 21}{1 + 0.5 + 0.25} = \frac{41.75}{1.75} \approx 23.9.$$

つまり $T_p \approx 23.9$ °C。一番近いセル (25 °C, 重み 1) に最も引っ張られ、遠いセルほど効きが弱いのが数字で見えます。もし節点がどれかのセルに重なると ($d \rightarrow 0$) その重みが無限大になり、そのセル値がそのまま採用されます (= 最近傍フォールバック)。

■次の FEM 式に出てくる記号 ε_{th} は熱ひずみ、 $\mathbf{I}$ は 3×3 単位テンソル、 Ω_e は要素 e の領域、 dV は体積積分の微小体積です。 B は節点変位からひずみを作る行列、 D はひずみから応力を作る弾性行列、 f_{th} は等価熱荷重 [N]、 K_s は構造剛性 [N/m] です。この B は `OI` の誤差共分散ではなく、この D は `POD` モード変位応答でもありません。テンソルの熱ひずみを実装の Voigt 表記に直すと、 $\alpha \Delta T (1, 1, 1, 0, 0, 0)^T$ です。後半 3 成分はせん断ひずみです。

7-2. 熱ひずみ → 節点荷重 → つり合い (熱弾性 FEM)

ここは `FrontISTR` が内部でやることです。私たちが渡すのは §7-1 の節点温度 と 材料定数 ($E, \nu, \alpha, T_{\text{ref}}$) ・底面固定の拘束だけ。以下は「その黒箱の中で何が起きているか」の説明で、自分でコードを書くわけではありません (返ってくるのは節点変位 u)。

節点温度が入ると、各要素に 熱ひずみ (等方・線膨張) が生じます:

$$\varepsilon_{\text{th}} = \alpha (T - T_{\text{ref}}) \mathbf{I}.$$

これは「温めると各要素がこれだけ伸びたい」という量。`FrontISTR` はこれを等価節点熱荷重に変換し、

$$f_{\text{th}} = \sum_e \int_{\Omega_e} B^T D \varepsilon_{\text{th}} dV$$

(B = ひずみ-変位行列、 D = 弾性行列)、底面固定の拘束のもとで線形弾性のつり合いを解きます:

$$K_s u = f_{\text{th}}$$

得られる $u = (u_x, u_y, u_z)$ が各節点の変位です。要するに「伸びたい量 $\alpha \Delta T$ を集めて、つながりと拘束の中でつり合わせると変位が出る」。片側だけ熱いと伸びに差が出て、円筒が反ります。

7-3. 変形アニメーション

表示の条件 :run/make_temp_to_disp_figs.py は、保存した1つの FrontISTR 変位形状を A - O 時系列の振幅で拡大縮小して動画にしています。色も元の変位場を使用します。各フレームで FrontISTR を解き直した変形時刻歴ではなく、反りの変化を説明する模式アニメーションです。定量的な時刻歴は次節のグラフで確認します。

t = 308 s (cooling (heater OFF))
deformation x6000

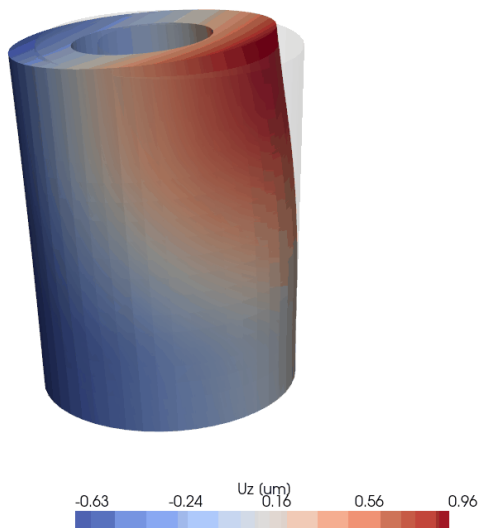


図 15: 熱変形のアニメーション

色 = U_z 、形 = FrontISTR の節点変位 (見やすさのため x6000 誇張)。加熱でヒータ側 (赤 = 上へ) が伸び、反対側 (青) と差がついて反る。300 秒でヒータ OFF → 全体が均されて反りが戻る。半透明の灰 = 元形状。

7-4. 変位の時系列グラフ

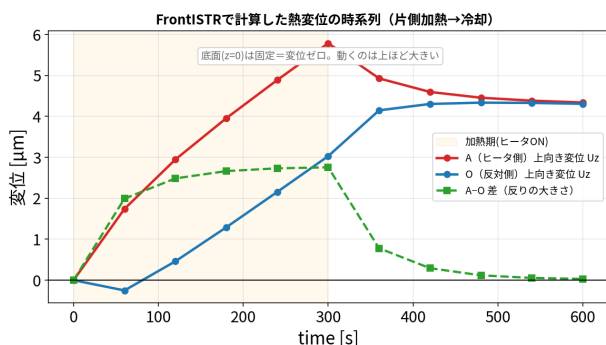


図 16: FrontISTR 真値の変位時系列

上面 2 点の上向き変位 U_z 。A (ヒータ側) が先に伸び、O (反対側) は遅れて追う。A - O 差 (緑) = 反りは加熱中に増えてピーク約 $2.8\mu\text{m}$ 、ヒータ OFF 後は温度が均されて差が消える。底面 ($z=0$) は固定なので変位ゼロ、動くのは上ほど大きい。

同化の精度を確認するときは、同じ量に ROM 真値温度を FrontISTR の変位画像へ通した参照値を黒太線で重ねます。次の図は真値 (黒) と、温度だけ/温度+変位を観測したデータ同化後の推定値 (色線) を同じ $U_z(A) - U_z(O)$ で比較したものです。薄い帯は 5 seed のばらつきです。図内に「FrontISTR 真値」とある場合も、この比較の真値は ROM 温度から FrontISTR のモード応答を用いて計算したものを指します。

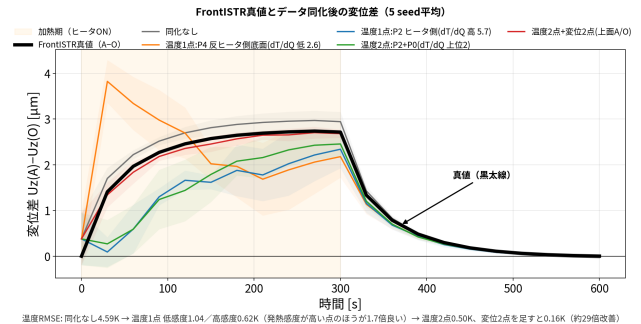


図 17: ROM 真値とデータ同化後温度から求めた変位差

黒太線は ROM 真値から同じ変位画像で計算した参照値。独立した OpenFOAM 温度履歴を毎時刻 FrontISTR で解析した真値ではありません。赤 (温度 2 点 + 変位 2 点) は真値に最も近く、温度観測だけ (緑) は変位差に誤差が残る。これは変位を状態変数として直接積分した結果ではなく、同化後温度を gappy-POD で全場へ戻し、その温度を FrontISTR の熱弾性画像へ通して得た推定値です。

この図の正直な読み: 変位差 RMSE で比べると、温度センサの場所 (低感度 1.19 vs 高感度 $0.99 \mu\text{m}$) や数 (2 点で $0.93 \mu\text{m}$) の差はわずかです。桁で効くのは変位観測の追加 ($0.17 \mu\text{m}$ = 約 6 倍改善)。これは「Q/h への温度感度が空間的にほぼ一様」という直接感度の結果とも整合します。これは今回の配置・ノイズ・モデルでの結果です。一般に温度センサの場所より数が重要だと結論することはできません。(なお温度そのものの RMSE では高感度 0.62 vs 低感度 1.04 K と約 1.7 倍の差があり、場所の効きは評価量に依存します。)

さらに大事な注意 — 「差」だけで評価してはいけない。個別の $U_z(A) \cdot U_z(O)$ も見ると:

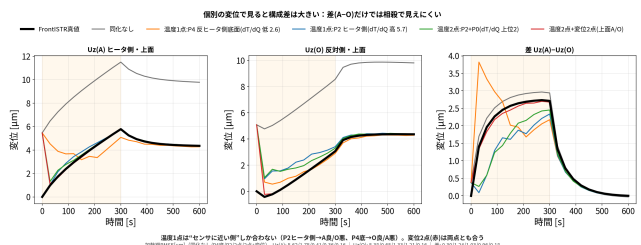


図 18: 個別の $U_z(A)/U_z(O)$ と差の比較

「差」には罫があります。同化なし (灰) は個別には $10\mu\text{m}$ 級で大外れなのに、差の RMSE は $0.30\mu\text{m}$ と最小級 (A・O が同じ方向に外れて相殺されるため)。個別で見ると、温度 1 点は「センサに近い側」しか合いません (P2 ヒータ側 → A 良 0.41 / O

悪 $1.33\mu\text{m}$ 、P4 底→O 良 $0.65 / A$ 悪 $1.75\mu\text{m}$ 。変位 2 点（赤）だけが両点とも $0.16\mu\text{m}$ で合う。→教訓：変位を合わせたい場所には変位観測を置く。そして評価は差だけでなく個別の変位でも行う。

7-4b.【重要】知りたい場所を測らずに当てる 一循環を断つ
§7-4 のグラフは $U_z(A) - U_z(O)$ （上面の反り）を評価していますが、「温度 2 点 + 変位 2 点」の構成では **A と O** をそのまま観測に使っています。

測った場所の値を当てて「当たりました」と言っているだけで、これは循環です。

■実務ではむしろ逆になる 工作機械で本当に知りたいのは工具先端側の変位です。ところが

- 加工中の工具先端に変位計は置けません
- 置けるのは機械の外側・静止部・冷えている部分

つまり実務は「測れる場所で測って、測れない場所を推定する」という構図です。そこで、この構図をそのまま実験にします。

	点	座標	扱い
評価点	A, O	上面 $z=100.5\text{ mm}$	本当に知りたい場所。工具先端側に相当
観測点	温度 P2, P0	$(36.5, 6.9, 50.8) / (21.3, -1.6, 51.6)\text{ mm}$	側面に貼った熱電対
観測点	B, C	中高さ $z=75.4\text{ mm}$	25 mm 下。ここら変位計を置ける

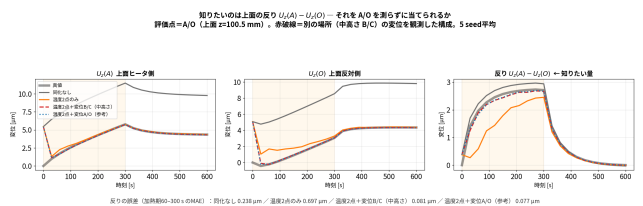


図 19: A/O を測らずに A/O の反りを当てる

■結果（知りたい量＝上面の反り。真値の振幅 $2.735\mu\text{m}$ ）

観測構成	反りの誤差	$U_z(A)$ 誤差	$U_z(O)$ 誤差
同化なし	$0.238\mu\text{m}$ ※	$5.614\mu\text{m}$	$5.376\mu\text{m}$
温度 2 点のみ	$0.697\mu\text{m}$	$0.160\mu\text{m}$	$0.848\mu\text{m}$
温度 2 点 + 変位 B/C（中高さ）	$0.081\mu\text{m}$	$0.055\mu\text{m}$	$0.027\mu\text{m}$
温度 2 点 + 変位 A/O（参考・循環あり）	$0.077\mu\text{m}$	$0.059\mu\text{m}$	$0.019\mu\text{m}$

（加熱期 60–300 s の MAE、5 seed 平均）

■3 つの読み取り ① 知りたい場所を測らなくても、ほぼ同じ精度で当たる

中高さ B/C を測るだけで反りの誤差 **$0.081\mu\text{m}$** （振幅の 3.0 %）。知りたい場所そのものを測った場合（ $0.077\mu\text{m}$ ）とほぼ同じです。

「工具先端に変位計を置けなくても、置ける場所に置けばよい」—実務で意味があるのはこの一点です。

② 温度だけでは反りは当たらない

温度 2 点だけだと $0.697\mu\text{m}$ （振幅の 25 %）。個別の $U_z(A)$ は $0.160\mu\text{m}$ まで合うのに、反りは合いません。 $U_z(A)$ と $U_z(O)$ の

差を当てるには、変位観測が必要です。

③ ※ 同化なしの **$0.238\mu\text{m}$** に騙されないこと

同化なしは個別に **$5.6\mu\text{m}$** も外れているのに、A と O が同じ方向に外れるため差では相殺されて小さく見えます。 $U_z(A) \cdot U_z(O)$ の個別グラフ（図の左・中）を見れば、灰色の線が真値から大きく離れているのが分かります。

「差」だけで評価してはいけないという §7-4 の注意が、こ
こでも効いています。

■循環していないと言える根拠

循環の条件	本実験
評価点（A/O）を観測に使ったか	使っていない（観測は温度 P2・P0 と変位 B/C）
真値を同化に渡したか	渡していない（ノイズ付きの 4 つだけ）
評価点と観測点は離れているか	25 mm 離れている（ $z=100.5$ と $z=75.4$ ）

■なぜ当たるのか 同化が直しているのは変位ではなく温度場だからです（§7-0）。

B/C の変位残差 → 共分散 $\text{Cov}(T, u)$ → 温度 5 点と Q を修正

↓
温度場が正しくなる

↓
 $u = W(T - T_{\text{ref}})$ で、どの点のどの方向も従う

↓
A/O の反りも正しくなる

A/O を当てているのではなく、温度場を当てた結果として A/O が付いてくるわけです。

■なぜ「反り」を知る必要があるのか 一工具長で補正量が変わる 「3 軸機は軸をオフセットするだけなので、傾きは補正できないのでは」という疑問があります。文献で確かめたところ、半分正しく、半分違います。

正しい部分：工具の姿勢（傾きそのもの）は 3 軸機では変えられません。

違う部分：傾き角は工具先端では並進の位置ずれに化けるので、NC オフセットで補正できます。

$$\delta \approx \theta \times L \quad (\theta: \text{傾き角}, L: \text{工具長})$$

Yang らはまさにこれを指摘しています。

“The present error compensation technology of computer numerical control machine tools **ignores radial thermal tilt angle errors of the spindle**, while the thermal-induced offset is **closely related to the tilt angle and the handle length**.” — Yang, Mei, Zhao, Ma, Shi, Feng (2015), *Proc. IMechE Part B* doi:10.1177/0954405414556499

彼らはヨー角・ピッチ角・伸びの 3 モデルを立て、傾き角と工具長に基づいて補正しています。

■本ケースで具体的に計算すると A と O は 57.4 mm 離れているので、反り 2.735 μm は傾き角に直すと

$$\theta = \frac{2.735 \times 10^{-6}}{57.4 \times 10^{-3}} = 4.77 \times 10^{-5} \text{ rad} = 0.0027^\circ$$

これを工具長で工具先端の位置ずれに換算します。

工具長 L	工具先端の位置ずれ $\delta = \theta L$	本研究の推定誤差
50 mm	2.38 μm	0.070 μm
100 mm	4.77 μm	0.140 μm
150 mm	7.15 μm	0.211 μm
200 mm	9.53 μm	0.281 μm
300 mm	14.30 μm	0.421 μm

同じ機械・同じ温度でも、工具を長くすると位置ずれが比例して大きくなります。100 mm の工具で 4.8 μm 、200 mm なら 9.5 μm 。傾き角を知らなければ、この工具長依存を計算できません。

これが「反りを推定する価値」です。並進成分だけ測っている、工具を替えるたびに補正量が変わることに対応できません。

■ただし機械によっては無視できる 旋盤 (turning center) を扱った Zhao らは、最大傾き角が 0.0099° と小さいため「多くの場合、主軸軸線の動きを並進運動とみなしてよい」と結論しています。

— Zhao, Guo, Tang, Zhang, Yi (2025), *Scientific Reports* PMC12223237 (全文無料)

本ケースの 0.0027° はその 1/3.6 です。傾きが効くかどうかは、機械構成と工具長で決まります。本研究の値は「工具が長いほど効いてくる」領域にあります。

■手続き (再現用)

- 評価点を決める : A, O (上面 z=100.5 mm) — 同化に一切使わない
- 観測点を決める : 温度 P2・P0 + 変位 B・C (中高さ z=75.4 mm)
- でたらめな初期温度・誤った Q, h から 60 メンバー
- 30 秒ごとに前進 → 2. の 4 つだけ観測して状態を更新 (20 回)
- 同化後の温度場から評価点 A/O の変位を計算
- A/O の真値と比べる

再 現: `OMP_NUM_THREADS=4 python3 run/eval_AO_from_elsewhere.py`

7-4c. 温度センサを 1 本も使わず、変位計だけで温度場を直せるか

「変位を観測して、データ同化で温度を直す」— これが本研究の方式です。では温度センサを一切使わず、変位計だけでも成立するのでしょうか。

実機では内部に熱電対を入れられないことがあります。変位計だけで足りるならセンサ構成の選択肢が広がるので、確かめました。

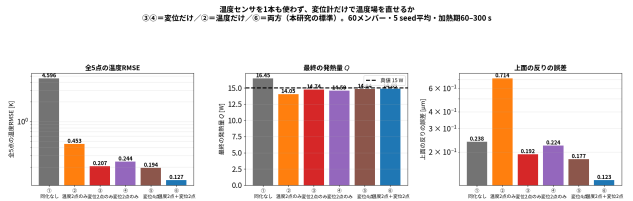


図 20: 変位だけで温度場を直せるか

観測構成	温度センサ	変位計	全 5 点の温度 RMSE	最終 Q	反りの誤差
① 同化なし	0	0	4.596 K	16.45 W	0.238 μm ※
② 温度 2 点のみ	2	0	0.453 K	14.05 W	0.714 μm
③ 変位 2 点のみ (上面 A/O)	0	2	0.207 K	14.74 W	0.192 μm
④ 変位 2 点のみ (中高さ B/C)	0	2	0.244 K	14.59 W	0.224 μm
⑤ 変位 4 点 (A/O + B/C)	0	4	0.194 K	14.84 W	0.177 μm
⑥ 温度 2 点+変位 2 点	2	2	0.127 K	14.89 W	0.123 μm

(60 メンバー・5 seed 平均・加熱期 60–300 s。真値 $Q = 15.00$ W)

☒ この表だけ集計窓が 60–300 s です (初回サイクルのスピンアップを除外)。本記事の他の表は 0–300 s (§7-6 の定義) なので、同じ構成でも数値が少し違います。温度 2 点 + 変位 2 点は 0–300 s だと 0.160 K、60–300 s だと 0.127 K です。

■結論 : 変位計だけで温度場は直せる。しかも温度計だけより良い ③ 変位 2 点だけ (温度センサ 0 本) で温度 RMSE 0.207 K。②温度 2 点だけ (0.453 K) の 2.2 倍良い結果になりました。

温度を測らずに温度を当てているわけですが、不思議なことではありません。§7-0 のとおり、変位計は「5 点温度の重み付き和」を測っています。重みは D の行そのもので、A の U_z なら (−0.0077, −0.0088, −0.0063, −0.0065, +0.0008)。1 本で 5 点すべてに触れているので、温度計 1 本より情報が広いのです。

■なぜ変位のほうが強いのか

	温度計	変位計
測るもの	1 点の温度	全 5 点の重み付き和
触れる範囲	1 点	5 点すべて
情報の種類	局所	大域

温度計 2 本では「P2 と P0 の値」しか分かりませんが、変位計 2 本は「5 点の重み付き和が 2 通り」分かります。未知が 7 個 (T_1, T_5, Q, h) なので、広く触れるほうが絞り込みが速いわけです。

■ただし組み合わせが最良 ⑥ 温度 2 点 + 変位 2 点が 0.127 K で最良です。③の 0.207 K より 1.6 倍良い。局所 (温度) と大域 (変位) は補完関係にあり、どちらか一方を増やすより組み合

わせたほうが効きます。

■実務的な読み取り

1. 内部に熱電対を入れられない機械でも、変位計だけで温度場を推定できる
2. 変位計は置ける場所でよい (③上面 0.207 K と ④中高さ 0.244 K でほぼ差がない)
3. ただし温度計を足せるなら足したほうがよい (0.207 → 0.127 K)

■※①の「反りの誤差 0.238 μm 」に注意 同化なしは個別には 5.6 μm も外れているのに、A と O が同じ方向に外れるため差では相殺されて小さく見えているだけです (§7-4b)。

再 現: OMP_NUM_THREADS=4 python3 run/disp_only_da_check.py

7-5. $x \cdot y \cdot z$ の全成分を見る 一観測していない方向・評価点はどうなるか

ここまでの図はすべて z 成分 (上下・軸方向) だけでした。FrontISTR は当然 U_x, U_y, U_z の 3 成分を返しているの、観測していない x, y 方向と評価点 (未観測点) がどうなるかを確かめます。

観測に使うのは今までどおり「温度 2 点 + 上面 A/O の U_z だけ」。 x, y は一切観測しませんし、下の B・C 点は一度も同化に使いません。

点	座標	役割
A	(+28.7, 0, 100.5) mm	上面ヒータ側。 U_z のみ観測に使用
O	(- 28.7, 0, 100.5) mm	上面反対側。 U_z のみ観測に使用
B	(+28.7, 0, 75.4) mm	評価点。同化に一切使わない
C	(- 28.7, 0, 75.4) mm	評価点。同化に一切使わない

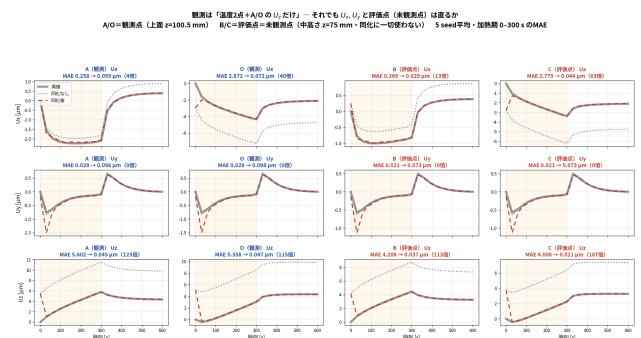


図 21: xyz 全成分の同化結果

行が成分 (上から U_x, U_y, U_z)、列が点 (A・O = 観測、B・C = 未観測)。灰点線 = 同化なし、赤破線 = 同化後、太い灰実線 = 真値。

5 seed 平均、初回サイクル ($t = 30$ s) のスピンアップを除いた 60–300 s の MAE:

点	区分	成分	真値振幅 [μm]	同化なし [μm]	同化後 [μm]	改善
A	観測	U_x	2.24	0.266	0.046	5.7 倍
A	観測	U_y	0.77	0.023	0.027	ほぼ変化なし
A	観測	U_z	5.78	5.614	0.031	179 倍
O	観測	U_x	4.36	2.873	0.036	80 倍
O	観測	U_z	4.36	5.376	0.032	166 倍
B	評価点	U_x	1.00	0.374	0.022	17 倍
B	評価点	U_z	4.45	4.217	0.024	172 倍
C	評価点	U_x	3.40	2.782	0.022	126 倍
C	評価点	U_z	3.27	4.022	0.021	195 倍

■分かったこと ① U_x は観測していないのに直る。O 点で 2.87 → 0.036 μm (80 倍)、未観測の C 点でも 2.78 → 0.022 μm (126 倍)。

理屈は §7-2 の熱弾性から明らかです。変位は温度場から $u = W(T - T_{\text{ref}})$ で決まり、 $W = K_s^{-1} H_T$ は 3 成分すべてを含む行列です。つまり同化が直っているのは変位ではなく温度場であって、温度場が正しくなれば W を通じて x も y も z も自動的に従います。「 U_z を観測して U_x が直る」のは不思議なことではなく、当然の帰結です。

② 評価点 (未観測点) は観測点と同じ率で直る。 U_z で観測点 179 倍・評価点 172 倍、 U_x で観測点 80 倍・評価点 126 倍。「測ったところだけ合っている」のではありません。これは §5-4 (未観測セルの温度復元) と同じ主張を、変位の側から確かめたことになります。

③ U_y だけは変わらない (0.023 → 0.027 μm)。これは失敗ではなく、同化なしの時点ですでに振幅 0.77 μm に対し誤差 0.023 μm (3%) で、直す余地がなかったためです。A・O・B・C はすべて $y = 0$ の対称面上にあり、非対称加熱が生む周方向変位がほぼ打ち消し合うためと考えられます (この解釈は未検証です。 $y \neq 0$ の点で確かめる必要があります)。

正直な注記: $t = 30$ s の初回サイクルだけを含めて平均すると、 U_y は 0.028 → 0.098 μm と「悪化」して見えます。これは 1 回目の更新での行き過ぎが平均を押し上げているだけで、2 サイクル目以降は真値に乗っています。上の表が 60–300 sなのはこのためです。

再 現: OMP_NUM_THREADS=4 OPEN-BLAS_NUM_THREADS=4 python3 run/eval_xyz_disp_da.py (FrontISTR を 6 回呼んで 3 成分の変位演算子を作り、結果を results/xyz_disp_da.json に保存)

仕上げに、まったく別の未観測点でも確認します。観測点 (赤/橙) から離れた未観測の温度 3 点 (緑) と変位 3 点 (紫) の位置はこちら:

観測点 (赤/橙) と未観測の評価点 (緑=温度3点、紫=変位3点)

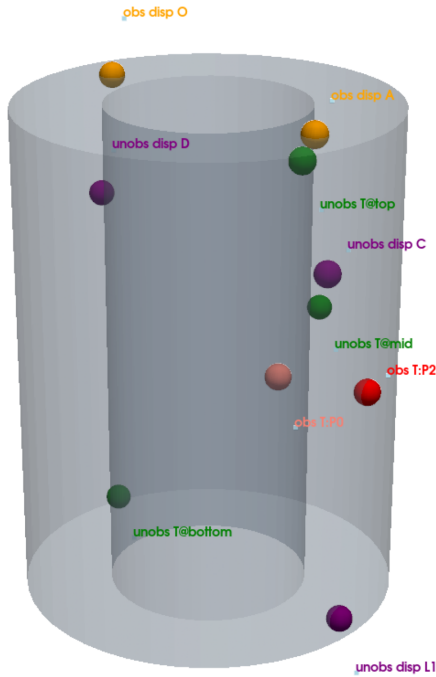


図 22: 観測点と未観測評価点の位置

この 6 か所すべてで、真値と各構成を時系列比較すると:

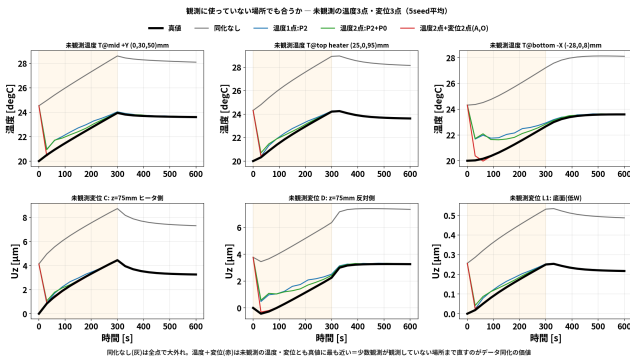


図 23: 未観測の温度 3 点・変位 3 点の時系列

同化なし (灰) は 6 点すべてで大外れ。同化した構成は観測していない場所でも真値に沿い、温度+変位 (赤) が最も近い (特に底面側の温度と $z=75\text{mm}$ 反対側の変位の立ち上がり)。少数の観測が、観測していない場所まで直す——これがデータ同化+POD 復元の価値です。

凡例の「高感度/低感度」の意味: ここでの感度は発熱量への温度感度 $\partial T / \partial Q$ です (q_{scale} を 1.0 → 1.1 に振った有限差分、加熱ピーク $t = 300\text{ s}$ で評価)。高感度 = $\partial T / \partial Q$ 最大の P2 (ヒータ側)、低感度 = 最小の P4 (反対側)、温度 2 点 (高感度) = 上位 2 ノード (P2 と P0)。上の観測点図 (sensor_positions) の色付けと同じ量です。なお変位センサ選びに使う熱感度 $W = K_s^{-1} H_T$ (温度→変位) とは別物なので注意 (温度センサは $\partial T / \partial Q$ 、変位センサは W で選ぶ、という役割分担)。

実務では、この温度→変位を毎回 FrontISTR で解く代わりに、線形化した感度 $W = K_s^{-1} H_T$ (blog_002 §4) で温度か

ら直接変位を出せます。 W は多数の温度入力に対する線形応答です。剛性行列の分解を再利用できますが、単一の熱荷重を 1 回解くだけで全 W が得られるわけではありません。POD 版では必要なモード応答だけを事前に計算します。

7-6. 観測構成を変えて比べる 一本研究の有用性

ここまでの部品 (POD + Q-DEIM + ROM + EnKF) を組み合わせ、観測の構成だけを変えて比べます。

先に読む: 106 のどの計算を説明しているか

106 では複数の観測構成を試しています。この節の主試験は **P2/P0 + 変位 A/O** (run/run_da_compare.py) です。これとは別に、**P2/P0 + 高 W 変位 S1/S2** を観測して、観測しない **C/D** を評価した試験があります。観測点・変位画像・ノイズの乱数系列が異なるため、Q の値や変位誤差を混せて読まないでください。

後者の実際の手順を、「今回行った計算を順番に追う」にまとめました。読む順番は次のとおりです。

1. **60 組** の仮説を用意する。各組は温度 P0~P4、発熱倍率 q 、放熱 h の 7 個を持つ。Q は $15q\text{ W}$ 。
2. 次の **30 秒** まで予測する。各組の $q \cdot h$ で RK4 を進め、その組の予報温度を得る。
3. 温度から変位を予測する。5 点温度 → POD 係数 → 事前計算した FrontISTR 応答 D の順に計算する。
4. 測定値との差を作る。予報は $[T_2, T_0, u_{S1}, u_{S2}]$ の 4 成分。C/D は観測に入れない。
5. **60 組** の共分散を作る。同じ時刻の平均と偏差から、 7×4 の C_{zy} と 4×4 の C_{yy} を 59 で割って計算する。
6. 温度 $\cdot q \cdot h$ を補正する。カルマンゲインで 4 成分の残差を 7 未知数へ配り、次の予測に使う。

例えば温度 P0 と発熱倍率 q の補正は、コードの添字で書くと

$$T_0^a = T_0^f + K_{0,0} d_{T2} + K_{0,1} d_{T0} + K_{0,2} d_{S1} + K_{0,3} d_{S2},$$

$$q^a = q^f + K_{5,0} d_{T2} + K_{5,1} d_{T0} + K_{5,2} d_{S1} + K_{5,3} d_{S2}.$$

ここで f は偏差膨張後の予報、 a は同化後、 d は「観測+メンバー別の観測摂動- 予報観測」です。各式の最後の 2 項が、変位観測から温度や発熱倍率へ入る補正です。補正した q を 15 倍すると Q [W] になります。

S1/S2 試験の最初の更新 (seed=20260913, 30 秒) を実数で再計算すると、変位 S1/S2 の残差による平均 Q への寄与は **+19.642 W**、**- 15.039 W** でした。温度からの寄与と足し合わせ、制約を適用すると Q 平均は **17.090 → 10.432 W** となります。以後の更新で修正され、この seed の 600 秒では **14.890 W** です。初回から真値に一致するわけではありません。

数値の内訳、観測ノイズと観測摂動の違い、実コード、保存ファイル、5seed 結果は手順の詳細に記載しています。

本研究で実際に行った手続き (run/run_da_compare.py)

ここに書くのは説明用の簡略例ではなく、表の数値を出した計算そのものの設定である。

真値 (双子実験) 校正済みの 5 点 ROM を、発熱量 $Q = 15 \text{ W}$ (ヒータ点 P2)、放熱 $h = 0.01535 \text{ W/K}$ 、0–300 s 加熱・300–600 s 冷却で時間積分したものを真値とする。

状態ベクトル (7 次元)

$$z = (T_1, \dots, T_5, q, h), \quad Q = 15 q \text{ [W]}$$

q は発熱の倍率 (真値 $q = 1$)。変位は状態に入れない。

観測 (30 s ごと、計 20 回) 温度 2 点 (発熱感度 $\partial T / \partial Q$ の上位 2 点 = P2, P0) と、上面 A・O の U_z の計 4 個。真値にノイズを足して作る。

$$y = (T_{P2}, T_{P0}, U_z(A), U_z(O))^{\text{true}} + \varepsilon, \quad \varepsilon \sim N(0, R), \quad R = \text{diag}(0.3^2, 0.3^2, 0.3^2, 0.3^2)$$

(温度は K、変位は μm)

初期アンサンブル (60 メンバー) 真値を知らない状態から始めるため、広くばらつかせる。

成分	初期分布
$T_1..T_5$	一様分布 $[T_{\text{air}} - 3, T_{\text{air}} + 12] \text{ K}$
q	一様分布 $[0.3, 1.8]$ ($Q = 4.5 \sim 27 \text{ W}$)
h	正規分布 $N(0.02, 0.01^2)$ 、 $[10^{-3}, 0.1]$ に制限

各サイクルの手続き ($t_k = 30k \text{ s}$, $k = 1, \dots, 20$)

1. 予報 各メンバーを、そのメンバー自身の q, h を使って ROM で 30 s 前進させる (RK4、刻み 2 s)。 q, h は前進中は変えない。
2. 予報観測 Y_f (60×4) 各メンバーについて、温度 2 点はそのまま取り出し、変位 2 点は温度から計算する (§7-0)。

$$u^{(m)} = u_{\text{mean}} + D U_{r,P}^+ (T_5^{(m)} - \bar{T}_P)$$

3. 共分散膨張 メンバーの平均からのずれを 1.02 倍に広げる (標本数不足で分散が縮みすぎるのを防ぐ)。
4. ゲイン 60 メンバーの偏差 dZ (60×7)、 dY (60×4) から

$$C_{zy} = \frac{dZ^T dY}{59} (7 \times 4), \quad C_{yy} = \frac{dY^T dY}{59} (4 \times 4), \quad K = C_{zy} (C_{yy} + R)^{-1} (7 \times 4)$$

5. 更新 メンバーごとに観測ノイズ $\varepsilon^{(m)} \sim N(0, R)$ を足して更新する (摂動観測法)。

$$z^{(m)} \leftarrow z^{(m)} + K(y + \varepsilon^{(m)} - Y_f^{(m)})$$

6. 制約 $q \in [0, 3]$ 、 $h \in [10^{-4}, 0.2]$ に収める。

これを 5 つの乱数シード (20260913~20260917) で繰り返し、平均を取る。

発熱量 Q が同定される仕組み 手順 5 で更新されるのは z の

7 成分すべてであり、 q はその 6 番目の成分である。 q の更新量は K の 6 行目

$$K_{q,:} = \text{Cov}(q, Y_f) (C_{yy} + R)^{-1}$$

で決まる。手順 1 で q の大きいメンバーほど温度が上がり、手順 2 でその温度から計算される変位も大きくなるため、 $\text{Cov}(q, T)$ と $\text{Cov}(q, u)$ は 0 にならない。したがって q を直接観測していなくても、温度と変位の残差 $y - Y_f$ が q を動かす。

結果 20 回の更新後、 $Q = 14.89 \text{ W}$ (真値 15 W)。加熱期の全 5 点温度 RMSE は 0.160 K 、上面の変位差の誤差は $0.165 \mu\text{m}$ (下の表の最終行)。なお真値も同じ ROM で作っているため、モデル誤差のない条件での結果である。

この表の「RMSE」は何を計算した値か

数字を出す前に定義をはっきりさせます。同じ観測構成で乱数条件を 5 回変えて計算します。 $s = 1, \dots, 5$ が計算番号 (seed)、 $i = 0, \dots, 4$ が ROM の温度点、 $m = 1, \dots, 60$ が EnKF メンバーです。**5 seed** は再試行の数であり、**60** メンバーとは別物です。

まず各時刻の同化後アンサンブルから、点ごとの推定温度を出します:

$$\hat{T}_{s,i}(t_k) = \frac{1}{60} \sum_{m=1}^{60} T_{s,m,i}^a(t_k)$$

(同化なしの場合は、同じ初期アンサンブルを更新せずに時間発展させた平均を使います)

次に、観測点だけでなく全 5 点 (未観測点を含む) で真値との差を取ります:

$$e_{s,i,k} = \hat{T}_{s,i}(t_k) - T_i^{\text{true}}(t_k), \quad r_{s,k} = \sqrt{\frac{1}{5} \sum_{i=0}^4 e_{s,i,k}^2}$$

表の「加熱期の平均 RMSE」は、この空間 RMSE を時刻と seed で平均した値です:

$$\overline{\text{RMSE}}_T = \frac{1}{5} \sum_{s=1}^5 \left[\frac{1}{10} \sum_{k=1}^{10} r_{s,k} \right] [\text{K}], \quad t_k = 30k \text{ s}$$

集計は **30, 60, ..., 300 秒の 10 回**の同化後です。0 秒は初期誤差が全構成で共通なので除外しています。

これは何ではないか全 20,696 セルの温度 RMSE でもなく、メンバーの標準偏差でもなく、最終 300 秒だけの誤差でもありません。また「全時刻・全点の二乗誤差をまとめてから平方根を取る」定義とも違います。

小さな数値例: ある時刻の 5 点の誤差が $[1, -2, 0, 2, -1] \text{ K}$ なら $r = \sqrt{(1+4+0+4+1)/5} = \sqrt{2} = 1.414 \text{ K}$ 。これを 10 時刻それぞれで計算して平均し、さらに 5 seed を平均します。

実装は run/run_da_compare.py の run_cfg_seed() 内の np.sqrt(((Z[:, :NPT].mean(0)-Ttr[ci])**2).mean())、時間平均は heat=(tgrid>0)&(tgrid<=300) と results[n][0][heat].mean() です。時間と seed はどちらも算術平均なので、平均の順序は交換でき

ます。

結果

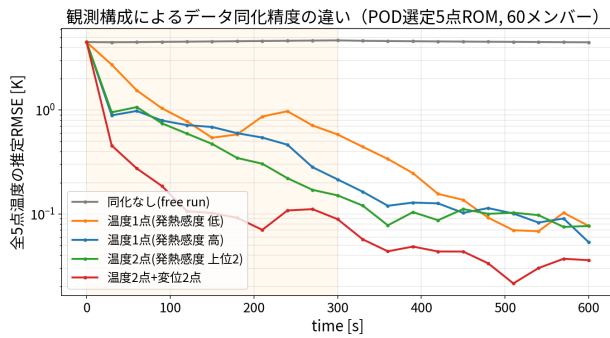


図 24: 収束の比較

ROM 双子実験 / 60 メンバー / 30 秒ごと 20 回更新 (0~600 秒) / 真値も同じ ROM / seed 20260913~20260917 の 5 回。

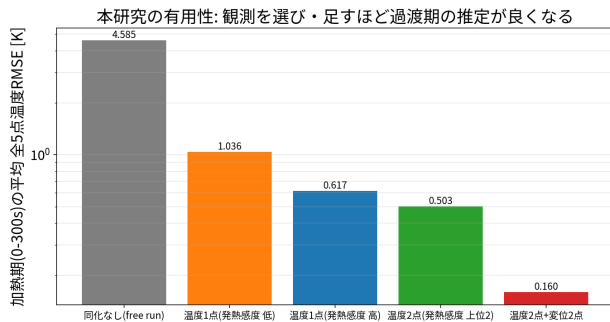


図 25: 有用性の棒グラフ

観測構成	加熱期の平均 RMSE	何が言えるか
同化なし (free run)	4.585 K	でたらのまま。データ同化なしでは推定できない
温度 1 点 (発熱感度 $\partial T/\partial Q$ 最小・P4)	1.036 K	同化すれば桁違いに改善。ただし感度の低い点は遅い
温度 1 点 (発熱感度 最大・P2)	0.617 K	同じ 1 点でも感度の高い場所に置くと 1.7 倍良い
温度 2 点 (発熱感度 上位 2・P2+P0)	0.503 K	センサを足すとさらに改善
温度 2 点+変位 2 点	0.160 K	変位を足すと決定的 (温度だけの 3 倍良い)

温度センサの選定基準は発熱感度 $\partial T/\partial Q$ です。熱感度 $W = K_s^{-1}H_T$ は変位センサの基準であって、温度センサに流用すると逆効果になります (検証: run/why_lowW_wins_check.py。温度 RMSE との相関は $\partial T/\partial Q - 0.835$ に対し $W+0.405$)。詳しくは blog_005 §4-2b。

8. まだ分かっていないこと 一代表点選択の未解決問題

ここまで「POD でモード→Q-DEIM で代表点→ROM 校正」ときれいに流れましたが、代表点の選び方には、実はまだ分かっていないことが残っています。正直に書いておきます。

1. 何点必要か (r の決め方) が「同化」目線では未説明。本記事はエネルギー基準 (2 モードで 99.9 % → 5 点) で決めましたが、これは場の復元の基準です。「データ同化の性能にとって最適な点数」は別問題で、センサ数を 1~5 点と変えたときの精度カーブの系統検証はこれからです。
2. Q-DEIM 点は「復元に良い点」であって「推定に最適な点」の保証はない。Q-DEIM は行列式最大化の貪欲近似で、大

域最適の保証がそもそもありません。さらに、推定したい量が変わると最適配置も変わります。実際、本リポジトリの実験では温度場推定なら「散らした」配置が Q-DEIM 点と同程度、Q・h 推定では温度センサの場所はほぼ効かない (感度が空間的にほぼ一様) という結果でした。「Q-DEIM で選んだ=観測もそこが最良」ではないのです。

3. 訓練条件への依存が未検証。代表点は訓練スナップショット (片側 15W 加熱・0-600 秒) から選ばれています。加熱位置や条件が変わったときに同じ 5 点で良いかは確かめていません (ROM の外挿問題と同根)。
4. 選ばれた点の物理的解釈は後付け。「底面 P3/P4 が選ばれたのは場を広くカバーするため」という説明は、出てきた結果を見ての解釈です。また校正された C_i, K_{ij} も「その 5 点で温度履歴に最良フィット」という意味しかなく、一意性 (別のパラメータ組でも同じ温度になる可能性) や物理的同定性は保証されていません ($\sum C_i$ が実物の熱容量とほぼ一致した、という傍証があるだけです)。
5. 実機の設置制約を考えていない。実機ではセンサを置けない場所 (内部・回転部など) があります。設置可能な領域に制限した上での選点 (制約付き Q-DEIM) は未実装です。

まとめると、「代表点 = ROM の状態を進める点」としては実証済み (残差 0.014K) ですが、「観測点としての最適性」「点数」「条件変化への頑健性」は開いた問題です。これらはそのまま、学会発表 (計算力学: 推定対象依存の観測配置) の研究課題になっています。

9. まとめ 一この記事でやったこと

1. EnKF は実ソルバだと重い → 予報モデルを軽い ROM に置き換える (単一 ROM の前進積分が約 31ms)。
2. 代表点は勘でなく POD + Q-DEIM でデータから選ぶ。
 - POD: スナップショット行列を SVD して場の「型」とエネルギーを抽出 ($XX^T\phi = \lambda\phi$)。
 - Q-DEIM: モード行列に列枢軸 QR をかけ、場を最もよく復元できる点を選定。
3. 小さな行列の手計算で、ランク 1 なら 1 点測るだけで場が完全復元できることを確認した ($\hat{u} = \bar{u} + U_r a$)。
4. 実データでは 2 モードで 99.9 %、選んだ 5 点 ROM は残差 0.014K で OpenFOAM を再現。
5. 温度が求まったら IDW で実形状へマッピング → 熱ひずみ $\alpha\Delta T \rightarrow K_s u = f_{th}$ で変位 (反り) が出る (§7)。

この軽い ROM の上で、OI (blog_002) と EnKF (blog_003) を回すと、温度場・発熱量 Q ・放熱係数 h を少数観測から高速に推定でき、さらに温度→変位まで通して「反り」を評価できます。「軽い予報モデル」と「賢い観測点」の 2 本柱がそろいました。

ただし忘れてはいけない点: ROM は OpenFOAM を本番ループでだけ置き換えるもの (§1-1)。モードも点も係数も CFD の結果から作った“蒸留物”で、CFD はオフラインで 1 回必ず要ります。そして訓練が覆った範囲でしか正しくない (形状・条件

が大きく変われば作り直し)。「高価な CFD を一度払って、安価なりアルタイムを得る」という前払い構造だ、と理解しておくのが大切です。

参考 (元になった実装・詳細)

- POD / Q-DEIM の数式と手続き : 本記事 §2~§4 (旧 01_pod_qdeim_algorithm.md を統合)
- 代表点選定 : run/select_points_qdeim.py
- ROM 構築・校正 : run/build_calibrate_rom.py, dacore/rom_general.py
- この上でのデータ同化 : blog_002_oi_data_assimilation.md, blog_003_ensemble_kalman_filter.md

研究として何を主張できるか、先行研究の確認範囲と優先する追加検証は研究の優位性と検証計画を参照してください。ROM 双子実験の精度と独立した OpenFOAM + FrontISTR 真値・実測での精度を区別しています。